
PSSP Virtual Notebook

Matteo Salvalaglio

Oct 01, 2025

CONTENTS

This is a collection of notes and exercises associated with the [Particulate Systems and Separations II module](https://www.ucl.ac.uk/chemical-engineering/), part of the second year ChemEng curriculum at UCL](https://www.ucl.ac.uk/chemical-engineering/).

These note are intended a complement to the lecture material available to UCL students.

Contribute to evolve these notes!

You can:

- Submit an issue (using the GitHub button, on the top)
- Suggest an edit (also using the GitHub button, on the top)
- Add a new notebook solving an exercise or developing a new topic (using GitHub and/or getting in touch via email at m.salvalaglio@ucl.ac.uk)

0.1 1. Material Balances in a Gas Permeation Unit

0.1.1 1.1 Operating Equation

Consider a binary gas phase mixture fed into a perfectly mixed membrane separation unit with molar flow rate F_{in} , and composition $\vec{z}$. The retentate stream is characterised by molar flow rate F_r , and composition $\vec{x}$, while permeate is characterised by molar flow rate F_p and composition $\vec{y}$.

Since we are dealing with a binary mixture, in all streams the compositions of A and B complement to 1 (e.g. for the permeate stream $y_B = 1 - y_A$). Hence we drop unnecessary subscripts and indicate with z , x , and y the molar fraction of component A in the mixture.

The global material balance on the unit reads: $F_{in} = F_p + F_r$

The material balance for component A reads: $F_{in}z = F_p y + F_r x$

Combining these two equations:

$$\begin{aligned} F_{in}z &= F_p y + (F_{in} - F_p)x \\ z &= \frac{F_p}{F_{in}}y + \left(1 - \frac{F_p}{F_{in}}\right)x \end{aligned} \quad (1)$$

the quantity $\frac{F_p}{F_{in}}$ is commonly defined as cut and indicated as θ .

Solving Eq.(?) for y leads to the expression:

$$y = \frac{(\theta - 1)}{\theta}x + \frac{1}{\theta}z \quad (2)$$

Eq. (2) is typically referred to as the Operating Equation of a perfectly mixed gas phase membrane module.

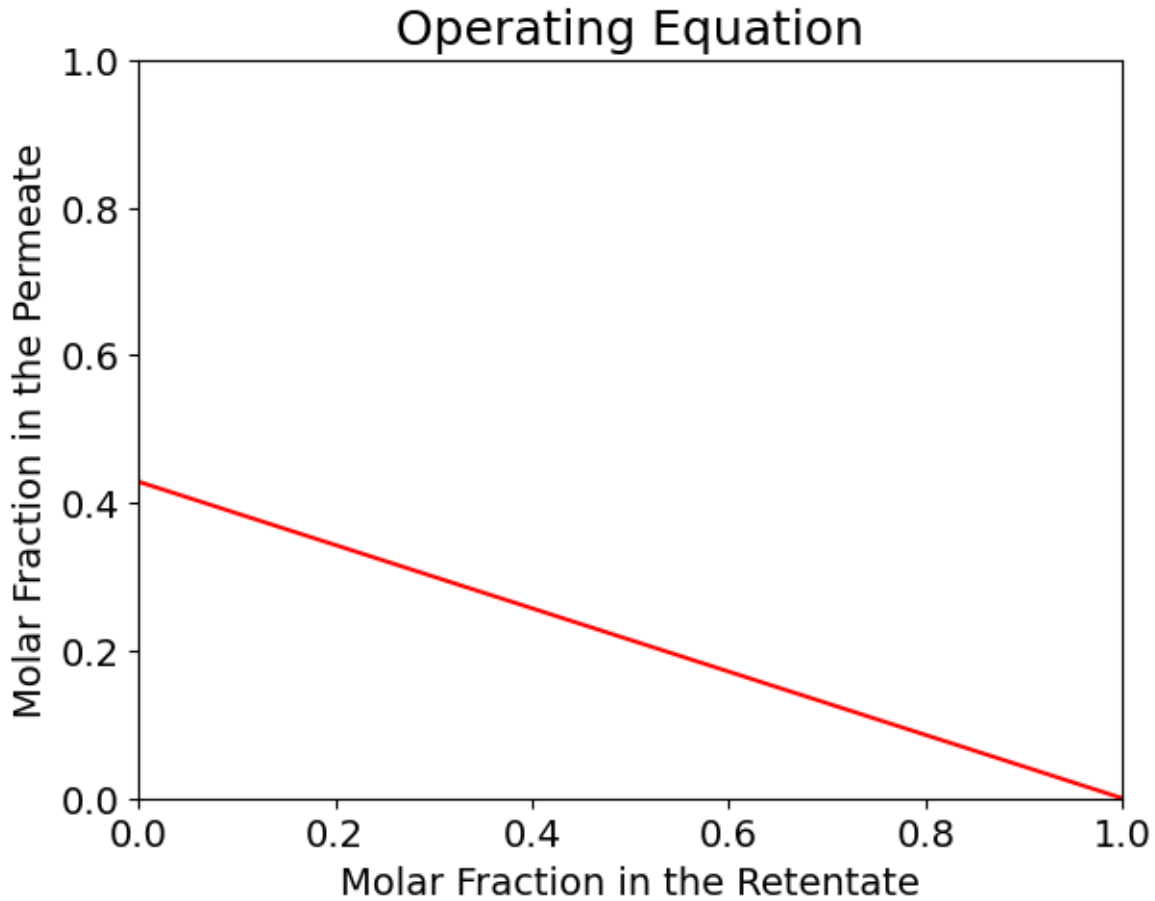
Graphical Representation

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters:
# cut
theta = 0.7
# molar fraction in the feed
z = 0.3
N = 100 #number of points
x_op = np.linspace(0, 1, N)

#Operating Equation
y_op = ((theta - 1) / theta) * x_op + (z / theta)

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,0.8,0.8])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
axes.plot(x_op,y_op, marker=' ', color='r')
axes.set_xlim([0,1])
axes.set_ylim([0,1]);
plt.title('Operating Equation', fontsize=18);
axes.set_xlabel('Molar Fraction in the Retentate', fontsize=14);
axes.set_ylabel('Molar Fraction in the Permeate',fontsize=14);
```



0.1.2 1.2 Rate Transfer Equation

Let us begin by considering that the permeate flow rate of component A is given by its molar flux through the membrane multiplied by the membrane total area:

$$F_p y = J \rho_A A_M \quad (3)$$

where J is the volumetric flux across the membrane, ρ_A is the molar density of component A, and A_M is the total area of the membrane. Applying the solution/diffusion model to define the volumetric flux Eq. (??) becomes:

$$F_p y = \frac{P_A}{l} \rho_A A_M (x p_r - y p_p) \quad (4)$$

where P_A is the permeability of the membrane to component A, l is the thickness of the membrane, p_r is the pressure on the retentate side, p_p is the pressure on the permeate side.

The same expression can be written for component B:

$$F_p (1 - y) = \frac{P_B}{l} \rho_B A_M ((1 - x) p_r - (1 - y) p_p) \quad (5)$$

Both Eq. (??) and (??) can be solved for F_p and equated, leading to:

$$\frac{P_A}{y} \rho_A (x p_r - y p_p) = \frac{P_B}{(1 - y)} \rho_B ((1 - x) p_r - (1 - y) p_p) \quad (6)$$

Eq. (??) can be solved either for y or for x , the latter being simpler. After some algebraic manipulation the solution for x is:

$$x = \frac{y \left[1 + \frac{p_p}{p_r} (1 - y) (\alpha_{AB} \frac{\rho_A}{\rho_B} - 1) \right]}{\alpha_{AB} \frac{\rho_A}{\rho_B} - (\alpha_{AB} \frac{\rho_A}{\rho_B} - 1) y} \quad (7)$$

where $\alpha_{A,B} = \frac{P_A}{P_B}$ is the ideal separation factor.

Eq. (??) can be simplified when the gas phase can be considered ideal. In such case $\rho_A = \rho_B = P/RT$, yielding:

$$x = \frac{y \left[1 + \frac{p_p}{p_r} (1 - y) (\alpha_{AB} - 1) \right]}{\alpha_{AB} - (\alpha_{AB} - 1) y} \quad (8)$$

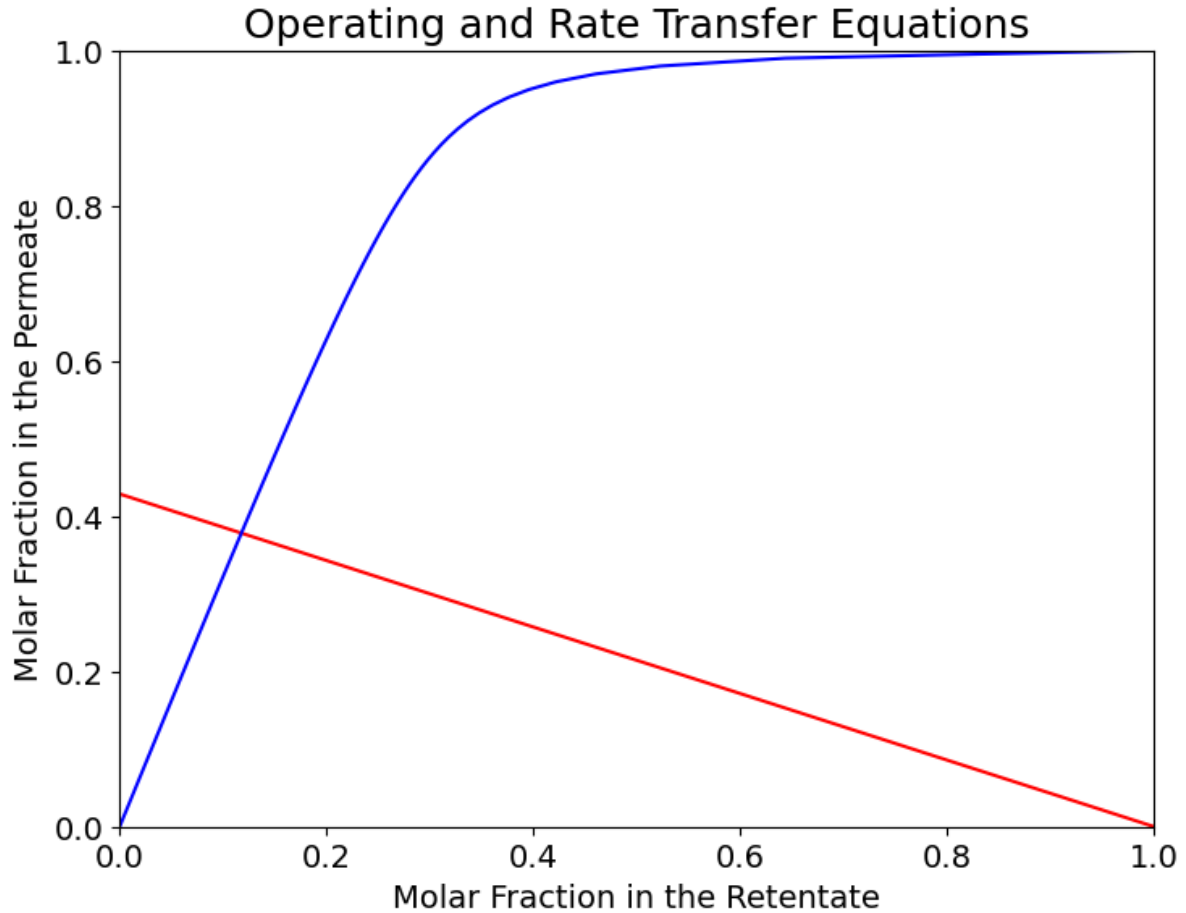
Eq. (??) is called *rate transfer equation* and together with the operating equation identify the membrane operating conditions in composition space.

Graphical Representation

```
# Parameters:
# Ratio between the permeate and retentate side pressures
Pp_Pr = 0.3
# ideal separation factor
alpha = 100
N = 100 #number of points
y_rt = np.linspace(0, 1, N)

#Rate Transfer
x_rt = y_rt * (1 + Pp_Pr * (1-y_rt) * (alpha - 1)) / (alpha - (alpha - 1) * y_rt)

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.0,1.0])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
axes.plot(x_op,y_op, marker=' ', color='r')
axes.plot(x_rt,y_rt, marker=' ', color='b')
axes.set_xlim([0,1])
axes.set_ylim([0,1])
plt.title('Operating and Rate Transfer Equations', fontsize=18);
axes.set_xlabel('Molar Fraction in the Retentate', fontsize=14);
axes.set_ylabel('Molar Fraction in the Permeate',fontsize=14);
```

The operating conditions correspond to the intersection of the RT and OP equations, i.e. the solution of a system of equations defined by the operating and rate transfer equations.

Numerical Solution

```
from scipy.optimize import fsolve

def equations(vars):
    x, y = vars
    eq1 = ((theta - 1) / theta) * x + (z / theta) - y
    eq2 = y * (1 + Pp_Pr * (1-y) * (alpha - 1)) / (alpha - (alpha - 1) * y) - x
    return [eq1, eq2]

x_set, y_set = fsolve(equations, (1, 1))

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.0,1.0])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
axes.plot(x_op,y_op, marker=' ', color='r')
axes.plot(x_rt,y_rt, marker=' ', color='b')
axes.plot(x_set,y_set, marker='o', color='lime', markersize=14)
```

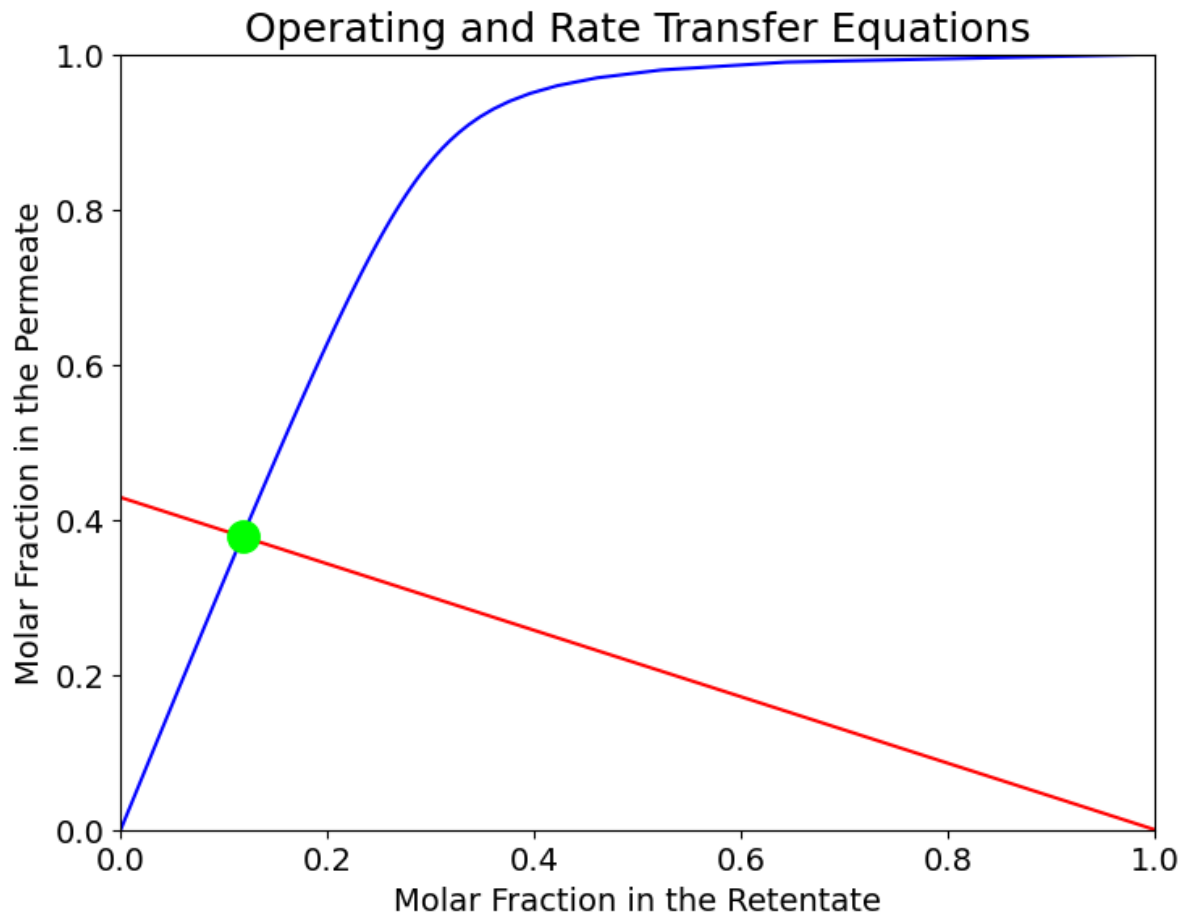
(continues on next page)

(continued from previous page)

```
axes.set_xlim([0,1])
axes.set_ylim([0,1])
plt.title('Operating and Rate Transfer Equations', fontsize=18);
axes.set_xlabel('Molar Fraction in the Retentate', fontsize=14);
axes.set_ylabel('Molar Fraction in the Permeate', fontsize=14);

print("Operating point -> x = ", x_set, " y = ", y_set)
```

Operating point -> x = 0.11767288696786887 y = 0.3781401912994848



0.1.3 Contributions

- Radoslav Lukanov, 16 Feb 2021
- Garv Mittal, 4 Feb 2024

0.2 2. Pervaporation

0.2.1 2.1 Vapour-Liquid Equilibrium in a Pervaporation unit

In pervaporation processes the feed/retentate is liquid, while the permeate is in the vapour phase. In this process, evaporation and permeation of different chemical through a semipermeable membrane take place simultaneously.

In order to tame the complexity of the processes taking place in this unit, a typical modelling assumption is that Vapour liquid equilibrium is established on the feed retentate side and permeation through the membrane takes place in the vapour phase.

Hence, pervaporation can be treated as a gas permeation problem by introducing a *virtual vapour phase* in equilibrium with the liquid on the retentate side of the membrane.

The driving force for a specie i can thus be written as the difference in the partial pressure for component i in this virtual vapour phase, and in the real vapour phase on the permeate side.

0.2.2 2.2 Vapour-Liquid Equilibrium conditions

To express the partial pressure of component i in a vapour phase in equilibrium with the liquid on the retentate side we shall begin by expressing the fundamental condition that has to be fulfilled by all species at thermodynamic equilibrium:

$$\mu_i^L(T, P, \vec{x}) = \mu_i^V(T, P, \vec{y}) \quad (1)$$

where $\mu_i^L(T, P, \vec{x})$ and $\mu_i^V(T, P, \vec{y})$ are the chemical potentials for specie i in the liquid and vapour phase, respectively.

For mixtures of ideal gases the chemical potential is conveniently expressed in differential form as a function of the partial pressure of component i , p_i as $d\mu_i = RT d \ln p_i$.

In analogy with this expression in the case of real gases fugacity f_i is introduced, leading to the differential expression $d\mu_i = RT d \ln f_i$. Integrating this expression between the phases in equilibrium yields:

$$\int_{L,T,P,\vec{x}}^{V,T,P,\vec{y}} d\mu_i = \int_{L,T,P,\vec{x}}^{V,T,P,\vec{y}} RT d \ln f_i$$

$$\mu_i^V(T, P, \vec{y}) - \mu_i^L(T, P, \vec{x}) = RT \ln \frac{f_i^V(T, P, \vec{y})}{f_i^L(T, P, \vec{x})} \quad (2)$$

Hence, the equilibrium condition expressed by Eq. (??) can be conveniently stated as:

$$f_i^V(T, P, \vec{y}) = f_i^L(T, P, \vec{x}) \quad (3)$$

The left-hand term can be written as:

$$f_i^V(T, P, \vec{y}) = \phi(T, P, \vec{y}) P y_i \quad (4)$$

where $\phi(T, P, \vec{y})$ is the fugacity coefficient, P the pressure and y_i the molar fraction in the vapour phase.

The right hand side term can instead be written as:

$$f_i^L(T, P, \vec{x}) = \gamma_i(T, P, \vec{x}) x_i \phi(T, P^o(T)) P^o(T) e^{\frac{v_i(P-P^o(T))}{RT}} \quad (5)$$

where x_i is the molar fraction of component i in the liquid phase, $\gamma_i(T, P, \vec{x})$ is the activity coefficient for component i in the liquid mixture, $P^o(T)$ is the equilibrium vapour pressure of the pure component i , $\phi(T, P^o(T))$ is the fugacity coefficient for the pure component i at T and $P^o(T)$, and the term $e^{\frac{v_i(P-P^o(T))}{RT}}$ is the Poynting correction. For a discussion on the origin of the Poynting correction please refer to the notes on osmosis.

Introducing Eq.(??) and Eq. (??) in Eq.(??), while considering negligible the Poynting correction, and ideal gas approximation applicable yields:

$$Py_i = \gamma_i(T, P, \vec{x})P^o(T)x_i \quad (6)$$

It should be noted that introducing the further hypothesis of ideal liquid mixture yields the Raoult law:

$$Py_i = P^o(T)x_i \quad (7)$$

0.2.3 2.3 Flux Equation in a pervaporation Unit

Considering now a binary mixture of components i and j , the flux equation for species i in a pervaporation unit is:

$$J_i = \frac{P_i}{l} (\gamma_i(T, P, \vec{x})x_i^r p_i^o(T) - p_p y_i^p) \quad (8)$$

where P_i is the permeability of species i through the membrane, l the thickness of the membrane, $\gamma_i(T, P, \vec{x})x_i^r p_i^o(T)$ is the partial pressure in the virtual vapour phase in equilibrium with the liquid on the retentate side, p_p the pressure on the permeate side, and y_i^p the molar fraction of component i in the real vapour phase on the permeate side.

The flux equation of component j can be expressed as a function of the molar fractions of component i taking advantage of the stoichiometric constraints $\sum_{i=1}^N x_i^r = 1$, $\sum_{i=1}^N y_i^p = 1$.

$$J_j = \frac{P_j}{l} (\gamma_j(T, P, \vec{x})(1 - x_i^r)p_j^o(T) - p_p(1 - y_i^p)) \quad (9)$$

0.3 3. The Osmotic Pressure

0.3.1 3.1 Osmosis

Osmosis is a mass transfer process that takes place spontaneously when two compartments, containing fluid mixtures of different composition are separated by a semipermeable membrane, i.e. a membrane that restricts the transfer of one of the components across compartments.

For the sake of simplicity, let us consider for now an ideal semipermeable membrane that completely restricts the transfer of the solute, while instead allowing the permeation of a solvent.

Let us consider an initial state characterised by two compartments (1 and 2), at the same temperature T and pressure P . Compartment 1 contains a binary mixture of component B (solute) in component A (solvent), e.g. NaCl in water. Compartment 2 instead contains a pure component A (e.g. pure water).

If the membrane is semipermeable and allows for the passage of the solvent alone the process of osmosis will lead to an equilibrium condition in which:

- Component A has transferred from compartment 2 to compartment 1.
- A pressure difference between compartments is established, where: $P_1 > P_2$.
- The pressure difference between compartments at equilibrium is the *osmotic pressure* $\pi = P_1 - P_2$.

In the next paragraphs we will see what is the thermodynamic origin of the osmotic pressure.

0.3.2 3.2 Equilibrium Conditions

In order to understand the origin of the osmotic pressure, i.e. pressure difference induced by a composition difference across compartments separated by a semipermeable membrane, we begin by defining the equilibrium state for the system described in the previous paragraph. To this aim we can rely on the thermodynamic definition of equilibrium for component A (e.g. water), i.e. the only component that can transfer across the membrane. Equilibrium is defined when A has the same chemical potential in both compartments

$$\mu_{A,1} = \mu_{A,2} \quad (1)$$

The equilibrium condition is conveniently written introducing fugacity. Fugacity and chemical potential are related by the expression $\mu_i = \mu_0 + RT \ln f_i$, where μ_0 is the chemical potential of an arbitrary reference state.

Equilibrium conditions are thus equivalently formulated as:

$$f_{A,1} = f_{A,2} \quad (2)$$

In compartment 1 we have a *non-ideal* mixture of two components. The fugacity of component A in compartment 1 is thus written as:

$$f_{A,1} = f_A(T, P_1, x_A) = \gamma_A(T, P_1, x_A) x_A f_A(T, P_1) \quad (3)$$

where:

- $\gamma_A(T, P_1, x_A)$ is the activity coefficient of specie A in the mixture
- x_A is the molar fraction of component A
- $f_A(T, P_1)$ is the fugacity of the *pure component* A at temperature T and pressure P_1

In compartment 2 we have instead a pure component A liquid, at temperature T and pressure P_2 . The fugacity of component A in compartment 2 thus simply corresponds to the fugacity of the pure component A at T, P_2 :

$$f_{A,2} = f_A(T, P_2) \quad (4)$$

The Poynting correction In order to define a common reference state between the two compartments we rewrite fugacity of the pure component A at pressure P_2 ($f_A(T, P_2)$) as a function of the fugacity of the pure component A at pressure P_1 and of the pressure difference ($P_2 - P_1$).

To do that we express the difference in chemical potential $\Delta\mu_{P_1 \rightarrow P_2}$ between pure liquid A at temperature T and pressure P_1 and pure liquid A at temperature T and pressure P_2 :

$$\Delta\mu_{P_1 \rightarrow P_2} = \int_{T, P_1}^{T, P_2} d\mu = \int_{T, P_1}^{T, P_2} v dP = v(P_2 - P_1) \quad (5)$$

where:

- the expression introduced for the definition of the $\Delta\mu$ is the Gibbs Duhem equation ($d\mu = -s dT + v dP$)
- v is the molar volume of the system

Using the definition of fugacity introduced earlier one can thus write:

$$\Delta\mu_{P_1 \rightarrow P_2} = RT \ln \frac{f_A(T, P_2)}{f_A(T, P_1)} = v(P_2 - P_1) \quad (6)$$

The fugacity of a pure A liquid at pressure P_2 can thus be written as:

$$f_A(T, P_2) = f_A(T, P_1) \left[\exp \left(\frac{v(P_2 - P_1)}{RT} \right) \right] \quad (7)$$

where the term $\left[\exp \left(\frac{v(P_2 - P_1)}{RT} \right) \right]$ is the Poynting correction.

0.3.3 3.3 The Osmotic Pressure

Given the definition of equilibrium conditions, the expressions of the fugacity of component A in the two compartments 1 and 2, and the Poynting correction given in the previous paragraph we can write:

$$\gamma_A(T, P_1, x_A)x_A f_A(T, P_1) = f_A(T, P_1) \left[\exp \left(\frac{v(P_2 - P_1)}{RT} \right) \right] \quad (8)$$

where $f_A(T, P_1)$ is the fugacity of the pure component A at pressure P_1 , now appearing on both sides of the equation.

We can thus divide both sides by $f_A(T, P_1)$ to obtain the expression:

$$\gamma_A(T, P_1, x_A)x_A = \left[\exp \left(\frac{v(P_2 - P_1)}{RT} \right) \right] \quad (9)$$

Rearranging this expression one obtains:

$$P_2 - P_1 = \frac{RT}{v} \ln (\gamma_A(T, P_1, x_A)x_A) \quad (10)$$

Recalling that the Osmotic pressure $\pi = P_1 - P_2$ we get:

$$\pi = -\frac{RT}{v} \ln (\gamma_A(T, P_1, x_A)x_A) \quad (11)$$

This expression **relates the composition of compartment 1**, containing a mixture of components A and B (e.g NaCl in water), **to the osmotic pressure** generated at equilibrium conditions when such compartment is in contact with a **semipermeable** membrane that has on the opposite side a compartment filled by pure component A liquid.

Please note that in this expression x_A is the molar fraction of the solvent (e.g. water). Since we are considering a binary system and from an engineering standpoint what is typically measured is the solute concentration we can rewrite the expression as follows:

$$\pi = -\frac{RT}{v} \ln (\gamma_A(T, P_1, x_A)(1 - x_B)) \quad (12)$$

Where now x_B is the molar fraction of component B (e.g. NaCl). When the solution in compartment 1 is diluted (x_B small), $\gamma_A(T, P_1, x_A) \rightarrow 1$, $\ln((1 - x_B)) \rightarrow -x_B$ thus:

$$\pi = -\frac{RT}{v} \ln((1 - x_B)) = RT \frac{x_B}{v} \simeq RT c_B \quad (13)$$

Where $c_B \simeq x_B v^{-1}$ is the molar concentration of component B, the solute.

3.3.1 Dependence of the Osmotic pressure on the composition of the system

Note that the expression $\pi = RT c_B$ is only valid for small molar fractions of component B, the solute.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters:
# molar fraction in the feed
N = 100 #number of points
x_B = np.linspace(0, 0.999, N)

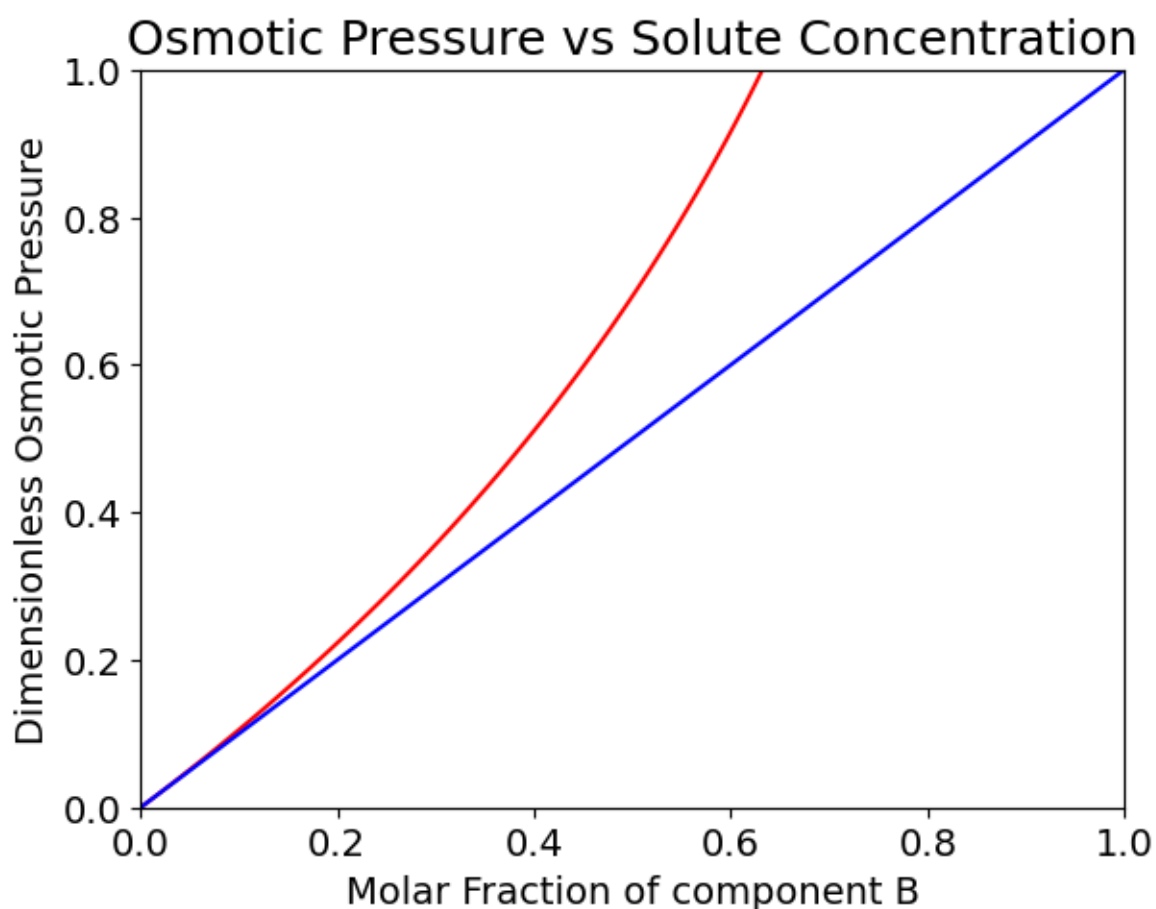
#Operating Equation
pi = - np.log(1-x_B)
```

(continues on next page)

(continued from previous page)

```
pi_simple = x_B

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,0.8,0.8])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
axes.plot(x_B,pi, marker=' ', color='r')
axes.plot(x_B,pi_simple, marker=' ', color='b')
axes.set_xlim([0,1]);
axes.set_ylim([0,1]);
plt.title('Osmotic Pressure vs Solute Concentration', fontsize=18);
axes.set_xlabel('Molar Fraction of component B', fontsize=14);
axes.set_ylabel('Dimensionless Osmotic Pressure',fontsize=14);
```



Note that the dimensionless osmotic pressure plotted above is obtained as $\pi v/RT$.

0.3.4 3.4 Flux of the solvent during Osmosis

In osmosis the flux of the solvent (component A) through a membrane is driven by the difference between the $\Delta P = P_1 - P_2$ across compartments and the osmotic pressure π :

$$J_A = p_A (\Delta P - \pi) \quad (14)$$

where p_A is the permeability of the membrane with respect to component A.

At equilibrium, when $\Delta P = \pi$ the net flux across compartments is null. Without adding any external pressure the spontaneous direction of the flux is from compartment 2 to compartment 1 ($J < 0$).

If we add a pressure, such that $\Delta P > \pi$ we induce the opposite flux, from compartment 1 to compartment 2. This process is named reverse osmosis.

```
import numpy as np
import pandas as pd
from matplotlib import cm
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Parameters:
# molar fraction in the feed
N = 100 #number of points
DeltaP = np.linspace(0, 1, N)
x_B = np.linspace(0, 0.8, N)
P_A = 1
DeltaP, x_B = np.meshgrid(DeltaP, x_B)

pi = - np.log(1-x_B)

#Flux Equation
J_A = P_A * (DeltaP - pi)

#Plotting

figure=plt.figure(figsize=(10, 8))
#axes = figure.add_axes([0.1,0.1,0.1,1.2,1.2,1.2])
axes = figure.add_subplot(projection='3d')

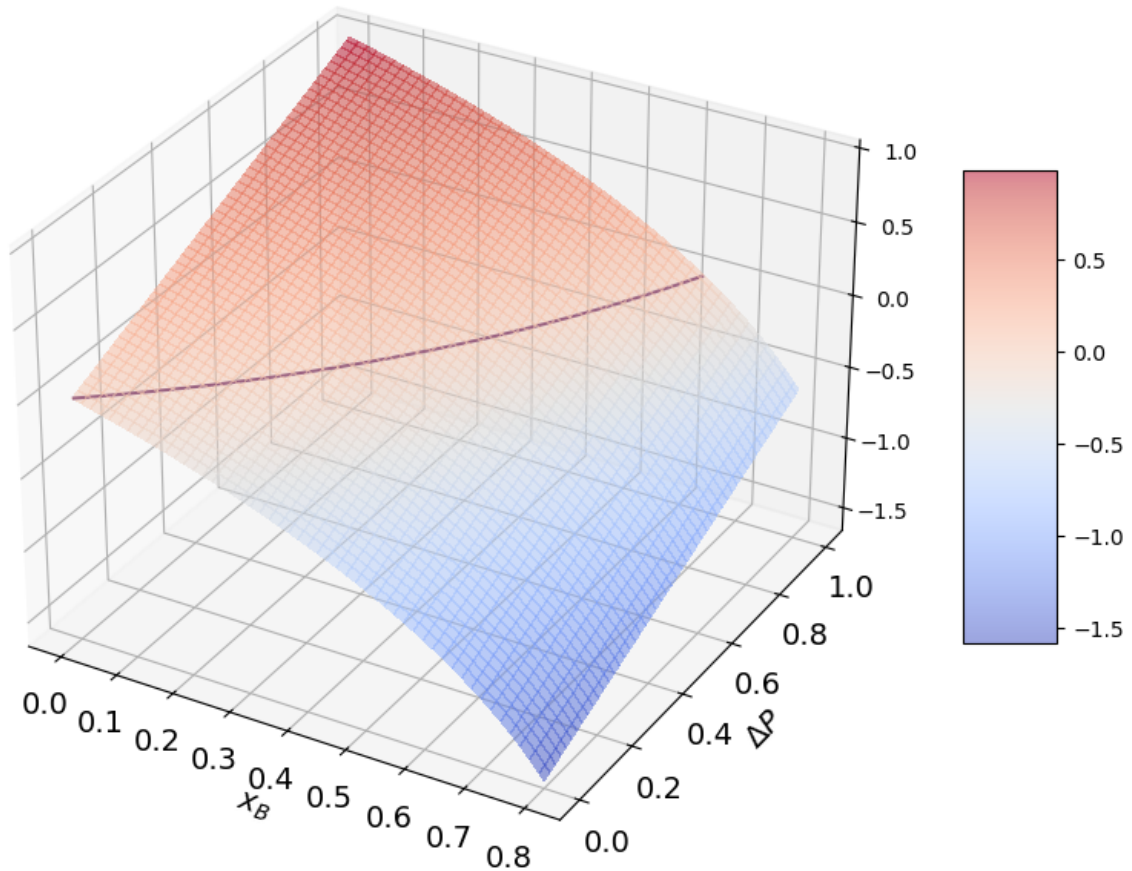
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

surf=axes.plot_surface(x_B,DeltaP,J_A,cmap=cm.coolwarm,
                      linewidth=0, antialiased=False,alpha=0.5)

contour=axes.contour(x_B,DeltaP,J_A,[0])

plt.title('Osmotic Flux vs  $\Delta P$  /  $x_B$ ', fontsize=18);
axes.set_xlabel('$x_B$', fontsize=14);
axes.set_ylabel('$\Delta P$', fontsize=14);
figure.colorbar(surf, shrink=0.5, aspect=5);
```


Osmotic Flux vs $\Delta P / x_B$



0.4 4. Membrane Processes Layouts

In this set of notes we will use an example ultrafiltration process to introduce material balances in different process configurations. We will consider a Batch, semi-Batch (feed and bleed) and continuous membrane processes to develop general ideas around material balances in membrane units. Every example will be introduced by a slight variant of a common problem statement.

0.4.1 4.1 Batch

Problem Statement

500 l of fruit juice are concentrated from an initial solid-residue content of 0.05 kg/l to a solid-residue content of 0.2 kg/l through a batch microfiltration process. The total area of the membrane is of 20 m² and the flux of pure water through the membrane is captured by the empirical expression: $J = BC^{-1}[m/h]$ with $B = 0.1$ where C is the solid residue concentration in kg/l. Compute the process time necessary to reach the target concentration.

Solution trace

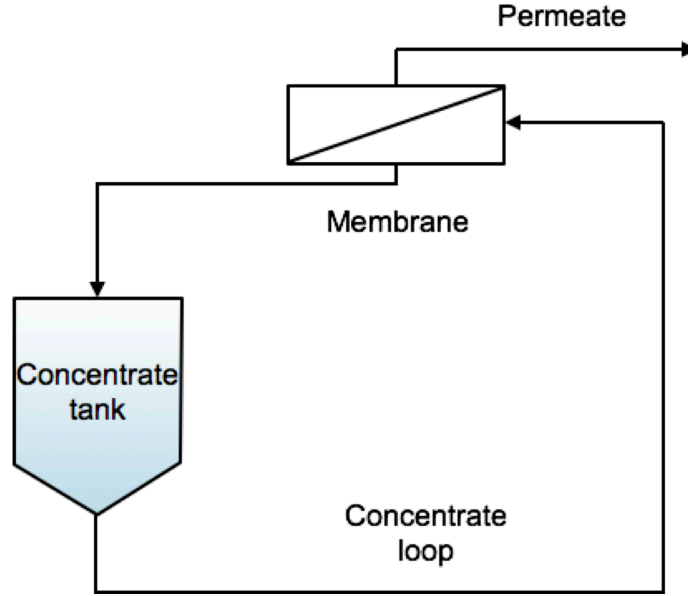


Fig. 1: Schematics of a Batch membrane process.

The global differential material balance (volume basis) reads:

$$\frac{dV}{dt} = -J A \quad (1)$$

where V is the volume in the concentrate loop, J the permeate flux per unit area and A the total membrane area. The permeate flux is defined by the expression:

$$J = BC^{-1} \quad (2)$$

with $B = 0.1$.

Since the solid residue never leaves the retentate loop, the differential material balance on the solid residue reads:

$$\frac{dm}{dt} = 0 \quad (3)$$

Which means that the total mass of solid residue is constant and equal to its initial mass:

$$m = m_0 = V_0 C_0 \quad (4)$$

The global differential material balance can thus be rewritten as:

$$\frac{dV}{dt} = -J A = -\frac{B}{V_0 C_0} A V \quad (5)$$

In this simple case it can be integrated analytically to compute the volume as a function of time:

$$\int_{V_0}^{V'} \frac{dV}{V} = \int_0^{t'} -\frac{B}{V_0 C_0} A dt \quad (6)$$

$$\int_{V_0}^{V'} \frac{dV}{V} = [\ln V]_{V_0}^{V'} = \ln \left(\frac{V'}{V_0} \right) \quad (7)$$

$$\int_0^{t'} -\frac{B}{V_0 C_0} A dt = -\frac{B}{V_0 C_0} A t' \quad (8)$$

Putting together both sides of the equation and substituting V' with V for the sake of simplicity in the notation we get:

$$V(t) = V_0 e^{-\frac{B}{V_0 C_0} A t} \quad (9)$$

which describes the time dependence of the volume in the batch membrane separator.

The concentration can thus be calculated as:

$$C(t) = \frac{m}{V(t)} = \frac{C_0 V_0}{V_0} e^{\frac{B}{V_0 C_0} A t} = C_0 e^{\frac{B}{V_0 C_0} A t} \quad (10)$$

The process time necessary to obtain a solid residue concentration of 0.2 [kg/l] is of 17.3 [h]

Numerical solution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

# Parameters:
N = 500 #number of points
time = np.linspace(0, 30, N)

#Every length in m

A=20; #m^2
B=0.1;
C0=0.05*1E3; #kg /l * dm^3/m^3
V0=500*1E-3; #l * m^3/dm^3

C_specific = 0.2*1E3

#Operating Equation
C = C0*np.exp(B*A/V0/C0*time)
V = V0*np.exp(-B*A/V0/C0*time)

def equation(proc_time):
    eq1 = C0*np.exp(B*A/V0/C0*proc_time) - C_specific
    return eq1

process_time = fsolve(equation,1)

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,0.8,0.8])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
axes.plot(time,V, marker=' ', color='b')

plt.title('Suspension Volume', fontsize=18);
axes.set_xlabel('time [h]', fontsize=14);
axes.set_ylabel('V [m$^3$]', fontsize=14);

figure=plt.figure()
axes = figure.add_axes([0.1,0.1,0.8,0.8])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
```

(continues on next page)

(continued from previous page)

```

axes.plot(time,C, marker=' ', color='r')
axes.plot(np.array([process_time[0],process_time[0]]),np.array([0, C_specific]),
↵marker=' ', color='lime', markersize=3)
axes.plot(np.array([0,process_time[0]]),np.array([C_specific, C_specific]), marker='
↵', color='lime', markersize=3)

axes.plot([process_time,process_time],[0, C_specific], marker=' ', color='lime',
↵markersize=3)

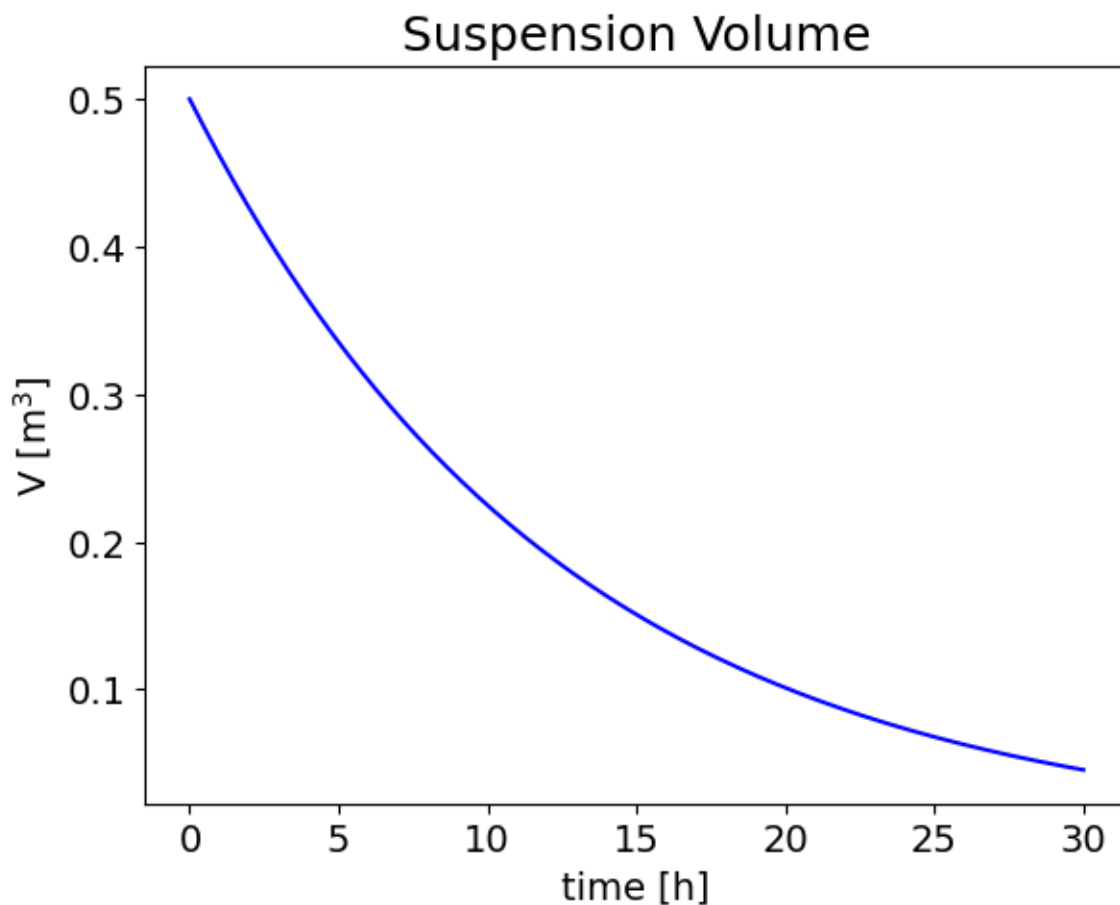
axes.plot(process_time,C_specific, marker='o' , color='lime', markersize=10)

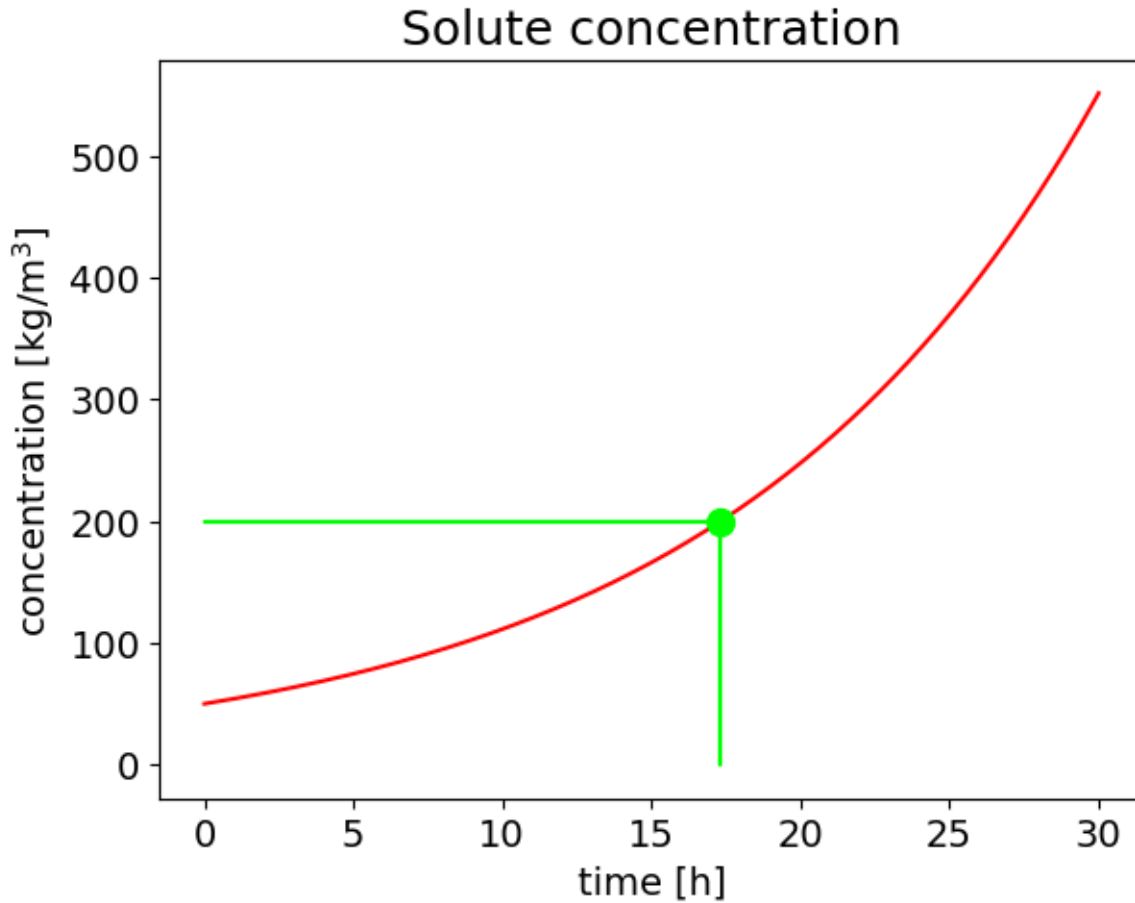
plt.title('Solute concentration', fontsize=18);
axes.set_xlabel('time [h]', fontsize=14);
axes.set_ylabel('concentration [kg/m$^3$]',fontsize=14);

print("The process time necessary to obtain a solid residue concentration of", C_
↵specific, "[kg/l] is:", process_time, "[h]")

```

The process time necessary to obtain a solid residue concentration of 200.0 [kg/l]↵
 ↵is: [17.32867951] [h]





0.4.2 4.2 Feed and Bleed

Feed and Bleed: Problem Statement

500 l of fruit juice are concentrated in 5 h of steady state operation from an initial solid-residue content of 0.05 kg/l through a feed and bleed process. The total area of the membrane is of 20 m² and the flux of pure water through the membrane is captured by the empirical expression: $J = BC^{-1}[m/h]$ with $B = 0.1$ where C is the solid residue concentration in kg/l. Is the steady state concentration of the retentate compatible with the specifics of 0.2 kg/l?

Feed and Bleed: Solution trace

Also in this case the global differential material balance and the solid-residue material balance can be written as follows:

$$\frac{dV}{dt} = Q_{IN} - J A - Q_{OUT} = 0 \quad (11)$$

$$\frac{dm}{dt} = Q_{IN}C_{IN} - Q_{OUT}C_{OUT} = 0 \quad (12)$$

At steady state $dV/dt = 0$ as well as $dm/dt = 0$. The two ODEs become then two algebraic equations that should be solved together to compute the steady state concentration. From the global material balance we get:

$$Q_{OUT} = Q_{IN} - J A = Q_{IN} - \frac{B}{C_{OUT}} A \quad (13)$$

and then, with some manipulations:

$$C_{OUT} = C_{IN} + \frac{AB}{Q_{IN}} \quad (14)$$

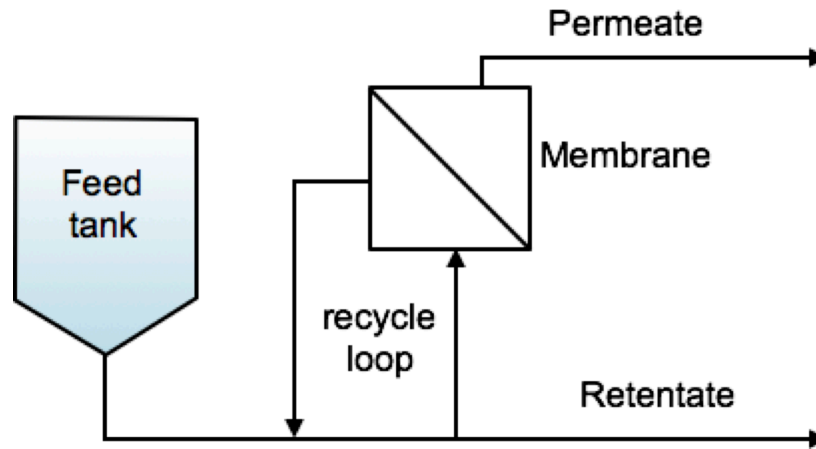


Fig. 2: Feed and Bleed process configuration scheme

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

# data:
A=20; # [m^2]
B=0.1;
C0=0.05*1E3; # kg / l * dm^3/m^3
V0=500*1E-3; # l * m^3/dm^3
process_time=5; # [h]

Cout=C0+A*B*process_time/V0

print("The steady state concentration is", Cout, "[kg/m^3]")

```

The steady state concentration is 70.0 [kg/m³]

0.4.3 Cascade configuration

Problem Statement

500 l of fruit juice are concentrated in 5 h of steady state operation from an initial solid-residue content of 0.05 kg/l through four membrane separation units characterised by a total membrane area of 20 m² each. The flux of pure water through the membrane is captured by the empirical expression: $J = BC^{-1}[m/h]$ with $B = 0.1$ where C is the solid residue concentration in kg/l.

Is it more efficient to design a single stage configuration with four units in parallel or a cascade configuration with four units in series?

Cascade: Solution trace

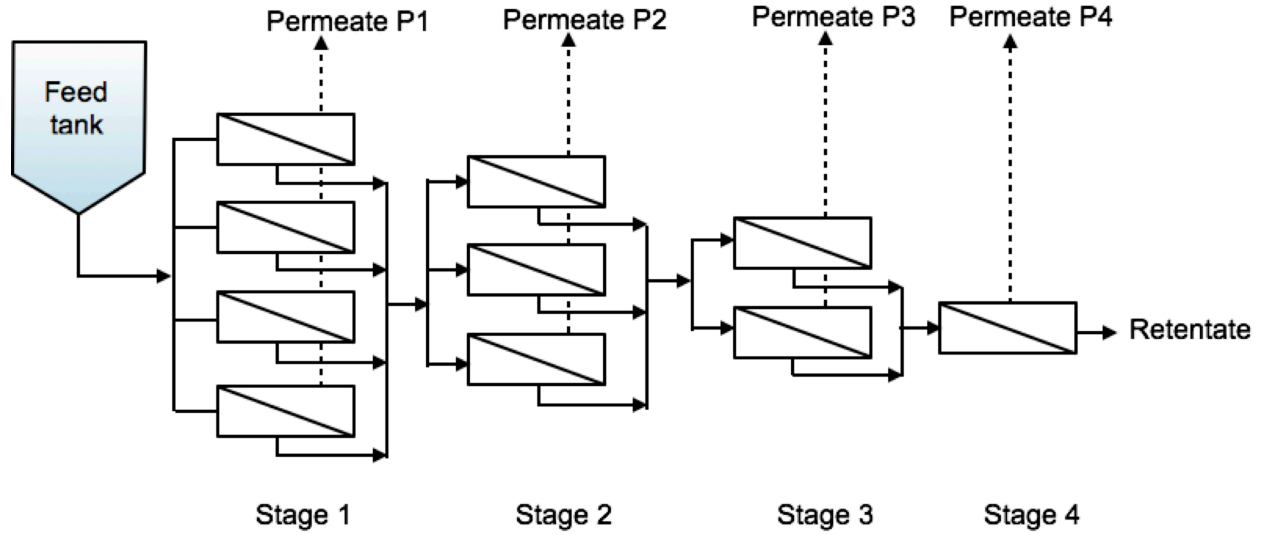


Fig. 3: Cascade configuration schematic representation.

Each stage can be treated like a single unit in which the feed stream corresponds to the retentate stream from the previous unit. We can thus define:

$$C_{OUT,i} = C_{IN,i+1} \quad (15)$$

$$Q_{OUT,i} = Q_{IN,i+1} \quad (16)$$

where the index i identifies the stage.

The material balances for each stage at steady state can thus be written as:

$$\frac{dV}{dt} = Q_{IN,i} - n_i J(C_{IN,i+1}) A - Q_{IN,i+1} = 0 \quad (17)$$

$$\frac{dm}{dt} = Q_{IN,i} C_{IN} - Q_{IN,i+1} C_{IN,i+1} = 0 \quad (18)$$

where n_i is the number of modules used in stage i .

For each stage we can thus compute the steady state concentration and concentrate volumetric flow by solving sequentially the following equations:

$$C_{IN,i+1} = C_{IN,i} + n_i \frac{AB}{Q_{IN,i}} \quad (19)$$

$$Q_{IN,i+1} = \frac{C_{IN,i} Q_{IN,i}}{C_{IN,i+1}} \quad (20)$$

with $i = 1 \dots N$ with N is the total number of stages.

Numerical Solution

The solution of a cascade composed of any number of stages, each formed by an arbitrary number of modules in parallel can be tackled sequentially through a simple cycle similar to the following, let's for example consider a system that reflects the sketch above i.e. with 4 stages implementing 4, 3, 2, and 1 modules each:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

# data:
A=20;
B=0.1;
C0=0.05*1E3; #kg /l * dm^3/m^3
V0=500*1E-3; #l * m^3/dm^3
process_time=5; # [h]

# The number of elements of this array corresponds to the number of stages.
# The value in each element is the number of modules per stage.
n=np.array([4,3,2,1]);

CIN=np.append(C0,np.zeros(np.size(n)-1))
FIN=np.append(V0/process_time, np.zeros(np.size(n)-1));

# Input to the intermediate stages
for i in range(1,np.size(n)):
    CIN[i]=CIN[i-1]+n[i-1]*A*B/FIN[i-1];
    FIN[i]=CIN[i-1]*FIN[i-1]/CIN[i];

# Output concentration
Cout=CIN[np.size(n)-1]+n[np.size(n)-1]*A*B/FIN[np.size(n)-1];

print("The steady state concentration is", Cout, "[kg/m^3]")

```

The steady state concentration is 720.7199999999999 [kg/m³]

In order to answer the problem request one should solve the system for two different configurations. In the first, representing a single-stage configuration with four membrane units in parallel, the number of stages should be set to $N = 1$ and the number of units in the first stage n_1 to 4.

```

# The number of elements of this array corresponds to the number of stages.
# The value in each element is the number of modules per stage.
n=np.array([4]);

CIN=np.append(C0,np.zeros(np.size(n)-1))
FIN=np.append(V0/process_time, np.zeros(np.size(n)-1));

# Input to the intermediate stages
for i in range(1,np.size(n)):
    CIN[i]=CIN[i-1]+n[i-1]*A*B/FIN[i-1];
    FIN[i]=CIN[i-1]*FIN[i-1]/CIN[i];

# Output concentration
Cout=CIN[np.size(n)-1]+n[np.size(n)-1]*A*B/FIN[np.size(n)-1];

print("The steady state concentration is", Cout, "[kg/m^3]")

```

The steady state concentration is 130.0 [kg/m³]

In the second the number of stages should be set to $N = 4$, each of the stages being assembled as a single unit ($n_i = 1$ for $i = [1, 4]$).


```
# The number of elements of this array corresponds to the number of stages.
# The value in each element is the number of modules per stage.
n=np.array([1, 1, 1, 1]);

CIN=np.append(C0,np.zeros(np.size(n)-1))
FIN=np.append(V0/process_time, np.zeros(np.size(n)-1));

# Input to the intermediate stages
for i in range(1,np.size(n)):
    CIN[i]=CIN[i-1]+n[i-1]*A*B/FIN[i-1];
    FIN[i]=CIN[i-1]*FIN[i-1]/CIN[i];

# Output concentration
Cout=CIN[np.size(n)-1]+n[np.size(n)-1]*A*B/FIN[np.size(n)-1];

print("The steady state concentration is", Cout, "[kg/m^3]")
```

```
The steady state concentration is 192.07999999999998 [kg/m^3]
```

The cascade configuration allows for a more efficient process since it allows to obtain a larger concentration with the same number of modules.

0.5 5. The Langmuir Isotherm

The Langmuir isotherm provides a relationship between the compositions of the fluid phase and of the adsorbed phase at equilibrium.

In the following we shall note with C the concentration of the component that undergoes adsorption in the fluid phase, and with q its molar fraction in the adsorbed phase.

The Langmuir isother is based on a set of assumptions:

- The kinetic constants of the adsorption and the desorption processes, as well as the equilibrium constant for the adsorption process, are independent from the fraction of free adsorption sites.
- Interactions between adsorbed molecules are negligible, or in other words the adsorption process does not depend on the local environment around an adsorption site.
- The surface of the adsorbent material can host a single monolayer of adsorbed molecules, hence its fraction of free sites θ_0 and the fraction of occupied sites Γ_1 are related through the stoichiometric relation $\Gamma_0 + \Gamma_1 = 1$.

In order to derive the langmuir adsorption isotherm from this set of hypotheses let's start by defining expressions for the adsorption and desorption rates.

The adsorption process takes place when a molecule of the component that is undergoing adsorption finds a free adsorption site. The rate of an adsorption event can thus be considered analogous to a bimolecular elementary reaction step with rate:

$$R_A = k_A \Gamma_0 C \quad (1)$$

Where k_A is the kinetic constant, Γ_0 is the fraction of free adsorption sites, and x is the molar fraction of the adsorbing component in the fluid phase.

The rate of desorption instead can be considered analogous to a monomolecular reaction, in which a (non-covalent) bond is broken and an adsorbed molecule is released to the fluid phase, leaving one free site. Its rate can be written as:

$$R_D = k_D \Gamma_1 \quad (2)$$

At equilibrium, the rates of adsorption and desorption are equal and the net flux of molecules to and from the adsorbed phase is null. We can thus write:

$$k_A \Gamma_0 C = k_D \Gamma_1 \quad (3)$$

Recalling the stoichiometric relation between the fraction of occupied and free sites Eq. \eqref{eq:3} becomes:

$$k_A (1 - \Gamma_1) C = k_D \Gamma_1 \quad (4)$$

which can be rearranged to provide an explicit expression of Γ_1 , the fraction of occupied sites, as a function of the concentration C in the fluid phase:

$$\Gamma_1 = \frac{k_A C}{k_D + k_A C} \quad (5)$$

To obtain a typical expression of the Langmuir isotherm we shall introduce as a single parameter for this equation the adsorption equilibrium constant $B_0 = k_A/k_D$:

$$\Gamma_1 = \frac{B_0 C}{1 + B_0 C} \quad (6)$$

0.5.1 5.1 Isotherm dependence on B_0

In order to understand the role of the adsorption equilibrium constant let us plot the fraction of occupied adsorption sites as a function of B_0

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# molar fraction in the feed
z = 0.3
N = 500 #number of points
C = np.linspace(0, 1, N)
B0 = np.array([0.1, 1, 2, 5, 10, 50, 100, 1000]);

color=iter(cm.gist_heat(np.linspace(0,1,np.size(B0))))

for i in range(0,np.size(B0)):
    c=next(color)
    #Langmuir isotherm
    Gamma1 = B0[i]*C / (1 + B0[i]*C)
    axes.plot(C,Gamma1, marker=' ', c=c)

plt.title('Langmuir Isotherm', fontsize=18);
axes.set_xlabel('C [mol l$^{-1}$]', fontsize=14);
axes.set_ylabel('$\Gamma_1$',fontsize=18);
```

```
<>:29: SyntaxWarning: invalid escape sequence '\G'
<>:29: SyntaxWarning: invalid escape sequence '\G'
/var/folders/4v/jxb311rn5nq2q59y91zzgxd40000gp/T/ipykernel_85059/3365319449.py:29:␣
↳SyntaxWarning: invalid escape sequence '\G'
    axes.set_ylabel('$\Gamma_1$', fontsize=18);
/var/folders/4v/jxb311rn5nq2q59y91zzgxd40000gp/T/ipykernel_85059/3365319449.py:29:␣
↳SyntaxWarning: invalid escape sequence '\G'
    axes.set_ylabel('$\Gamma_1$', fontsize=18);
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 import numpy as np
      2 import matplotlib.pyplot as plt
      3 from matplotlib.pyplot import cm

ModuleNotFoundError: No module named 'numpy'
```

0.5.2 5.2 Concentration in the adsorbed phase.

The Langmuir isotherm as discussed so far provides a relationship between the fractional occupation of adsorption sites Γ_1 and the concentration in the fluid phase C . This relationship is only dependent on the thermodynamics of adsorption, captured by the value of the adsorption equilibrium constant B_0 . In order to compute the concentration adsorbed by a specific material we need one additional parameter, the maximum concentration q_M in the adsorbed phase. The concentration in the adsorbed phase is thus computed as: $q = \Gamma_1 q_M$, hence:

$$q = q_M \frac{B_0 C}{1 + B_0 C} \quad (7)$$

The parameter q_M depends on the adsorbent material, and in particular on the amount of surface area per unit volume.

In the following we report the concentration in the adsorbed phase, in equilibrium with an arbitrary concentration C' in the fluid phase as a function of the affinity towards the stationary phase (captured by parameter B_0) and of its surface area (captured by parameter q_M).

```
import numpy as np
import pandas as pd
from matplotlib import cm
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Parameters:
# molar fraction in the feed
N = 100 #number of points
B0 = np.logspace(-3, 1.5, N)
q_M = np.logspace(1, 3, N)
C = 1 ; # arbitrary value of C'
B0, q_M = np.meshgrid(B0, q_M)

q=q_M * B0 * C / (1+B0 * C)

#Plotting

figure=plt.figure(figsize=(10, 8))
```

(continues on next page)

(continued from previous page)

```

axes = figure.gca(projection='3d')

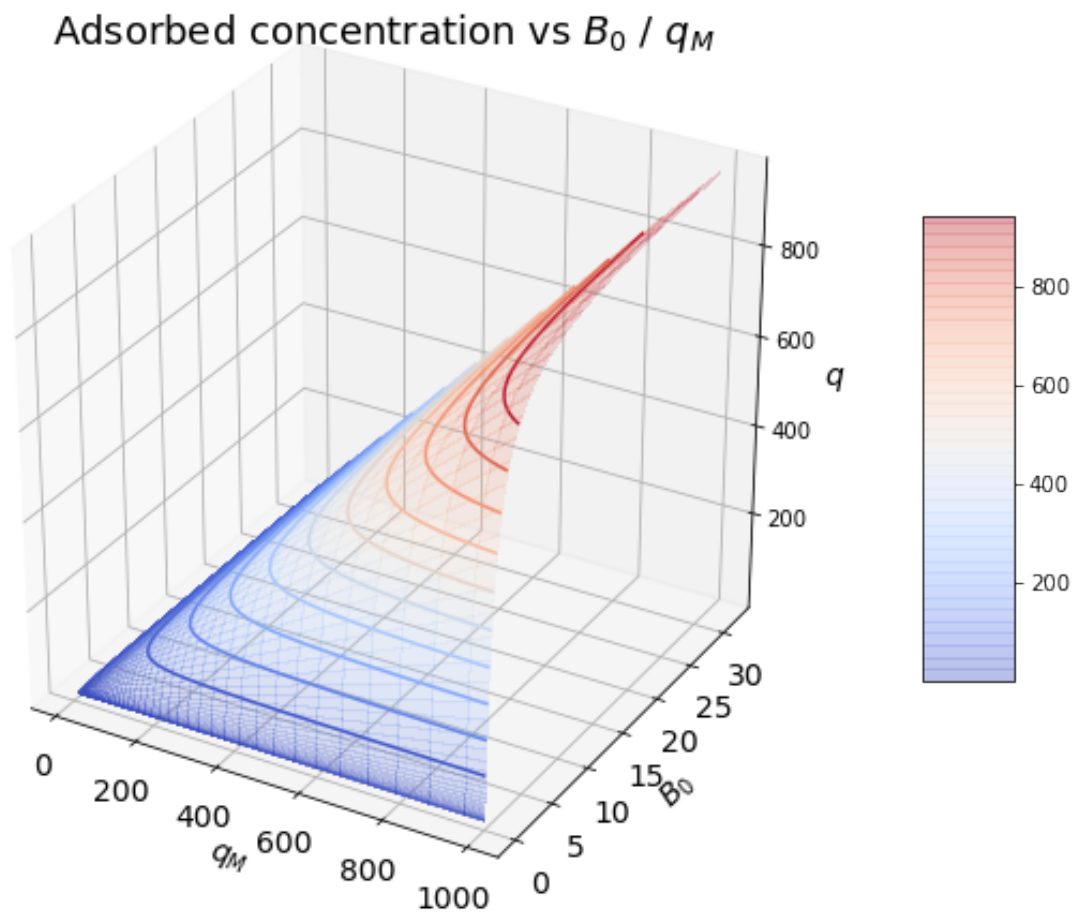
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

surf=axes.plot_surface(q_M,B0,q,cmap=cm.coolwarm,
                      linewidth=0, antialiased=False,alpha=0.2)

contour=axes.contour(q_M,B0,q,np.linspace(100, 800, 10),cmap=cm.coolwarm)

plt.title('Adsorbed concentration vs  $B_0$  /  $q_M$ ', fontsize=18);
axes.set_xlabel('$q_M$', fontsize=14);
axes.set_ylabel('$B_0$', fontsize=14);
axes.set_zlabel('$q$', fontsize=14);
figure.colorbar(surf, shrink=0.5, aspect=5);

```



0.5.3 5.3 Langmuir isotherm in the gas phase.

When describing the thermodynamics of adsorption from the gas phase it is convenient to introduce explicitly the pressure P in the Langmuir isotherm. In this case the expression of the monocomponent Langmuir isotherm becomes:

$$q = q_M \frac{B_1 P}{1 + B_1 P} \quad (8)$$

where $B_1 = B_0/RT$ and P is pressure.

0.5.4 5.4 Multicomponent, competitive adsorption

In multicomponent systems, in which N components compete for the same adsorption sites we can define the adsorption equilibrium constant of component i as:

$$B_{1,i} = \frac{\Gamma_i}{P_i \Gamma_0} \quad (9)$$

In a multicomponent system, the single layer hypothesis becomes:

$$\Gamma_0 + \sum_{i=1}^N \Gamma_i = 1 \quad (10)$$

Substituting in Eq. (??) the definition of adsorption equilibrium constant given in Eq. (??) we get:

$$\Gamma_0^{-1} = 1 + \sum_{i=1}^N B_{1,i} P_i \quad (11)$$

finally, substituting again Γ_0^{-1} in Eq. (??), and rearranging we get an expression of the fraction of sites occupied by specie i as a function of the partial pressure (concentration) of i in the fluid phase.

$$\Gamma_i = \frac{B_{1,i} P_i}{1 + \sum_{i=1}^N B_{1,i} P_i} \quad (12)$$

Finally, inserting the density of specie i in the adsorbed phase we get:

$$q_i = q_M \frac{B_{1,i} P_i}{1 + \sum_{i=1}^N B_{1,i} P_i} \quad (13)$$

where N is the total number of components, P_i is the partial pressure component i in the gas phase, and $B_{1,i} = B_{0,i}/RT$ where $B_{0,i}$ is the adsorption equilibrium constant for component i .

In the following we report the competitive adsorption isotherms of two components adsorbing from a binary mixture.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
```

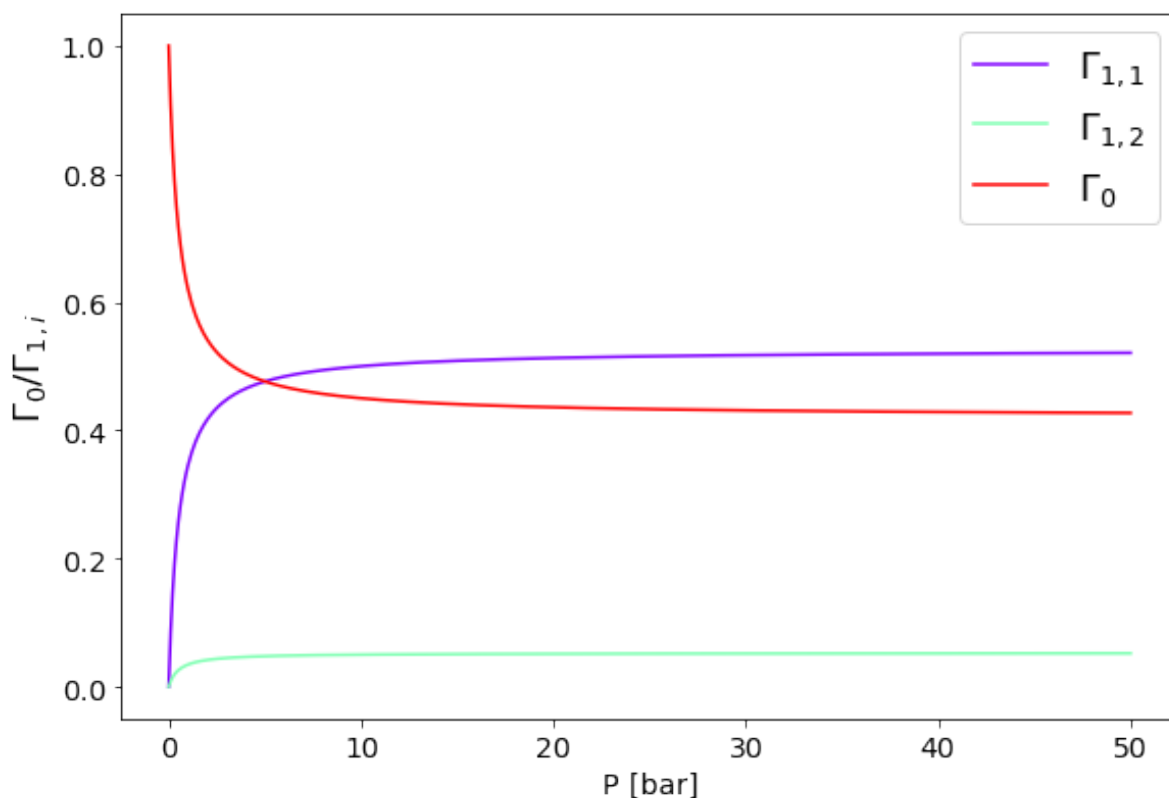
(continues on next page)

(continued from previous page)

```
# molar fraction in the feed
z = 0.3
N = 500 #number of points
P = np.linspace(0,50, N)
y = 0.1 #component 1 is in a smaller proportion
B11 = 10; #component 1 affinity for the stationary phase is higher than that of
#component 2
B12 = 1;

color=iter(cm.rainbow(np.linspace(0,1,3)))
c=next(color)
Gamma11 = B11*P*y / (1 + B11*P*y + B12*P*(1-y))
axes.plot(P,Gamma11, marker=' ', c=c)
c=next(color)
Gamma12 = B12*P*y / (1 + B11*P*y + B12*P*(1-y))
axes.plot(P,Gamma12, marker=' ', c=c)
c=next(color)
Gamma0 = 1 - Gamma11 - Gamma12
axes.plot(P,Gamma0, marker=' ', c=c)

axes.set_xlabel('P [bar]', fontsize=14);
axes.set_ylabel('$\Gamma_0 / \Gamma_{1,i}$', fontsize=18);
axes.legend(['$\Gamma_{1,1}$', '$\Gamma_{1,2}$', '$\Gamma_0$'], fontsize=18);
```



0.6 6. The BET Isotherm

The BET (Brunauer, Emmet and Teller) isotherm allows to extend ideas and hypotheses that are adopted to obtain the Langmuir isotherm to describe the adsorption of a component from a gas phase in cases in which it can form multiple layers on the surface of the stationary phase, up to the limit in which it condenses into a macroscopic liquid phase.

The hypotheses at the foundation of the BET theory are:

- Lateral interactions between adsorbed molecules (i.e. interactions between molecules adsorbed in the same layer) are negligible. In other words the adsorption process does not depend on the local environment around an adsorption site. This hypothesis is common to the derivation of the Langmuir isotherm.
- Molecules can adsorb in an infinite number of layers.
- The adsorption enthalpy on the first layer is constant and independent from the fractional occupation of that layer.
- The adsorption enthalpy in layers from the second onwards is constant and corresponds to the condensation enthalpy.

Following an approach similar to the one adopted for the derivation of the Langmuir isotherm let us begin by defining the rates of adsorption and desorption. In the case of the BET isotherm the rates will depend on the layer considered and will therefore have subscript i that indicate the layer on which adsorption takes place.

The adsorption rate on layer i can be written as:

$$R_{A,i} = k_{A,i} \Gamma_i P \quad (1)$$

The desorption rate instead, explicitly introducing the dependence on the adsorption enthalpy through an Arrhenius-like functional form, is written as:

$$R_{D,i} = k_{D,i} \Gamma_i = A_i e^{\frac{-E_i}{RT}} \Gamma_i \quad (2)$$

where E_i is the adsorption energy. Denoting with E_0 the adsorption energy on a free adsorption site, the adsorption energy on all successive layers is $E_2 = E_3 = \dots E_n = \lambda$, where λ is the condensation enthalpy.

Following the same criterion also the adsorption constant $k_{A,i}$, and the pre-exponential factor of the desorption constant A_i are independent from the layer index - except for those related to the adsorption on a free surface and the desorption from the first layer:

$$A_{i \geq 2} = \text{const} = A \quad (3)$$

$$k_{A,i \geq 1} = \text{const} = k_A \quad (4)$$

Having defined the rates of adsorption and desorption one can write population balances for every layer of the adsorbed phase. In the following Γ_0 represents the fraction of free adsorption sites, Γ_i represents the fractional occupation of the i^{th} layer.

$$\begin{cases} \frac{d\Gamma_0}{dt} &= -k_{A,0} \Gamma_0 P + k_{D,1} \Gamma_1 \\ \frac{d\Gamma_1}{dt} &= +k_{A,0} \Gamma_0 P - k_{D,1} \Gamma_1 - k_{A,1} \Gamma_1 P + k_{D,2} \Gamma_2 \\ \frac{d\Gamma_2}{dt} &= +k_{A,1} \Gamma_1 P - k_{D,2} \Gamma_2 - k_{A,2} \Gamma_2 P + k_{D,3} \Gamma_3 \\ &\vdots \\ \frac{d\Gamma_n}{dt} &= +k_{A,n-1} \Gamma_{n-1} P - k_{D,n} \Gamma_n - k_{A,n} \Gamma_n P + k_{D,n+1} \Gamma_{n+1} \\ &\vdots \end{cases} \quad (5)$$

At equilibrium the balances reduce to:

$$\begin{cases} 0 = -k_{A,0}\Gamma_0 P + k_{D,1}\Gamma_1 \\ 0 = +k_{A,0}\Gamma_0 P - k_{D,1}\Gamma_1 - k_{A,1}\Gamma_1 P + k_{D,2}\Gamma_2 \\ 0 = +k_{A,1}\Gamma_1 P - k_{D,2}\Gamma_2 - k_{A,2}\Gamma_2 P + k_{D,3}\Gamma_3 \\ \vdots \\ 0 = +k_{A,n-1}\Gamma_{n-1} P - k_{D,n}\Gamma_n - k_{A,n}\Gamma_n P + k_{D,n+1}\Gamma_{n+1} \\ \vdots \end{cases} \quad (6)$$

From the steady state balance on free sites we get:

$$k_{A,0}\Gamma_0 P = k_{D,1}\Gamma_1 \quad (7)$$

hence, inserting this equality into the balance on the fraction of sites occupied by a single layer, Γ_1 we get:

$$k_{A,1}\Gamma_1 P = k_{D,2}\Gamma_2 \quad (8)$$

Introducing the Arrhenius expression for the adsorption rates we get:

$$\Gamma_1 = \frac{k_{A,0}}{k_{D,1}}\Gamma_0 P = \frac{k_{A,0}}{A_1} e^{\frac{E_1}{RT}} P \Gamma_0 \quad (9)$$

and analogously for Γ_2 :

$$\Gamma_2 = \frac{k_{A,1}}{k_{D,2}}\Gamma_1 P = \frac{k_{A,1}}{A_2} e^{\frac{E_2}{RT}} P \Gamma_1 \quad (10)$$

a similar procedure can now be followed recursively to obtain a compact expression for the fraction of sites occupied by an arbitrary number of layers n . To this aim we can introduce two constants, α and β - defined as:

$$\alpha = \frac{k_{A,0}}{A_1} e^{\frac{E_1}{RT}} P \quad (11)$$

$$\beta = \frac{k_A}{A} e^{\frac{\lambda}{RT}} P \quad (12)$$

At this point the fraction of sites occupied by n layers can be obtained as:

$$\Gamma_n = \beta \Gamma_{n-1} = \beta^2 \Gamma_{n-2} = \dots = \beta^{n-1} \Gamma_1 = \beta^n \frac{\alpha}{\beta} \Gamma_0 \quad (13)$$

Deining for the sake of compactness the constant $B_2 = \alpha/\beta$ and introducing the volume of adsorbate per unit area v_s^1 one can express the total volume adsorbed as:

$$v_s = v_s^1 \sum_{i=1}^n i B_2 \beta^i \Gamma_0 = v_s^1 B_2 \beta \sum_{i=1}^n i \beta^{i-1} = v_s^1 B_2 \beta \frac{d \sum_{i=1}^n \beta^i}{d\beta} \quad (14)$$

By introducing the notable result for the term $\sum_{i=1}^n \beta^i$:

$$\sum_{i=1}^n \beta^i = \left(\frac{1 - \beta^{n+1}}{1 - \beta} \right) \beta \quad (15)$$

one gets:

$$\frac{v_s}{v_s^1} = \frac{B_2 \beta}{1 - \beta} \frac{[1 - (n+1)\beta^n + n\beta^{n+1}]}{[1 + (B_2 - 1)\beta - B_2 \beta^{n+1}]} \quad (16)$$

In order to obtain the final formulation of the BET isotherm it is useful to consider the limit corresponding to macroscopic condensation, which corresponds to a buildup of an infinite number of adsorbed layers, which corresponds to $\beta = 1$. Since this condition is realised for $P = P^o$, it can be shown that $\beta = \frac{k_A}{A} e^{\frac{\lambda}{RT}} P = P/P^o$.

Introducing this equality in eq16 one gets the so called limited form of the BET isotherm:

$$\frac{v_s}{v_s^1} = \frac{B_2 \left(\frac{P}{P^o}\right)}{1 - \left(\frac{P}{P^o}\right)} \frac{\left[1 - (n+1) \left(\frac{P}{P^o}\right)^n + n \left(\frac{P}{P^o}\right)^{n+1}\right]}{\left[1 + (B_2 - 1) \left(\frac{P}{P^o}\right) - B_2 \left(\frac{P}{P^o}\right)^{n+1}\right]} \quad (17)$$

note that the left hand side of this equation can be expressed in terms of unit mass of adsorbent, instead of unit surface.

0.6.1 6.1 Many isotherm shapes from the same expression

Depending on the parameters introduced in the BET isotherm its functional form can adapt to capture all known shapes of adsorption isotherm. In the following we shall demonstrate combination of parameters that produce isotherms belonging to all five of the classes introduced by Brunauer (Brunauer et al. JACS 1940).

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

N=100;

#Parameters
P0 = np.array([5, 12, 12, 5, 11, 11]);
B2 = np.array([10, 50, 2, 50, 2]);
n = np.array([1,500, 500, 5, 5]);

P = np.linspace(0, 10, N)
vv1 = np.zeros(np.size(P))

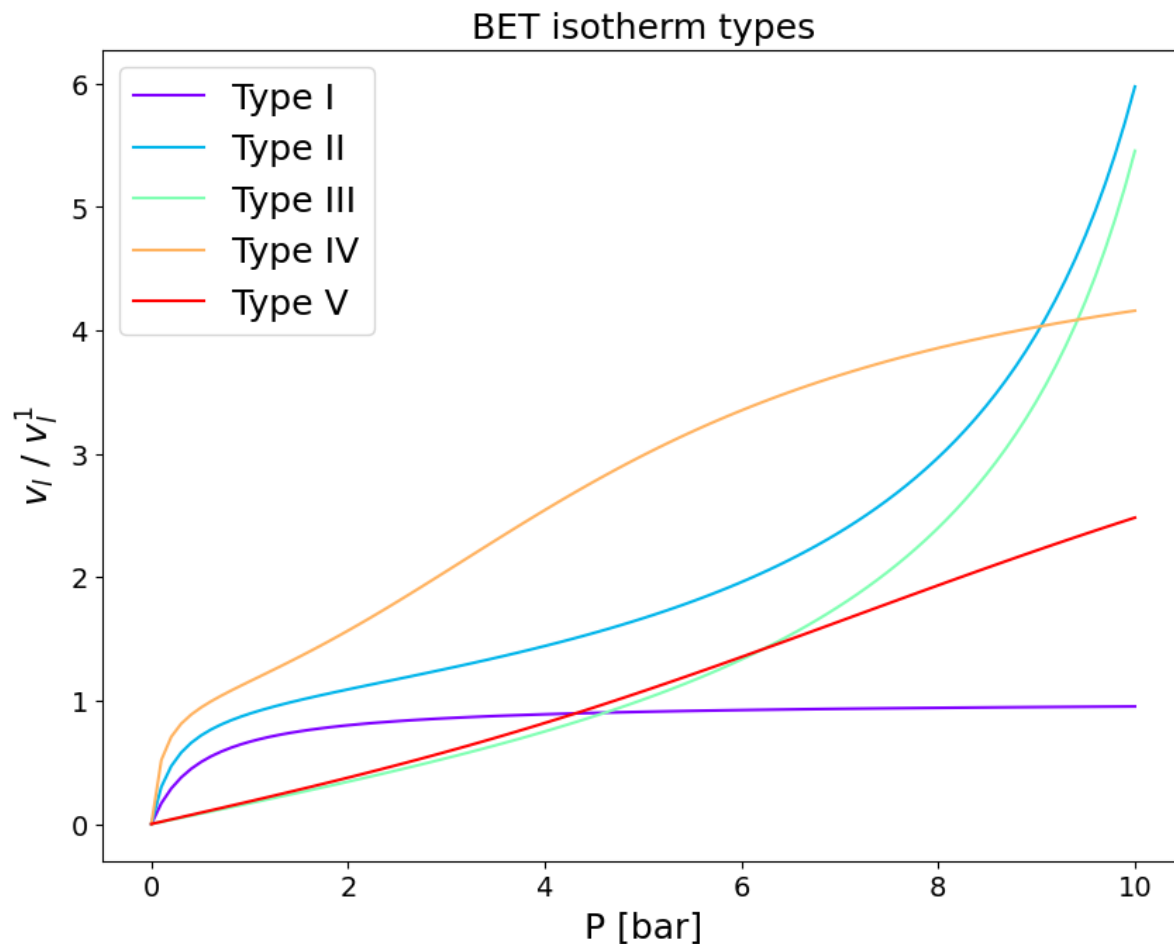
color=iter(cm.rainbow(np.linspace(0,1,np.size(B2))))

for i in range(0,np.size(B2)):
    S=P/P0[i]
    c=next(color)

    #Langmuir isotherm
    vv1=B2[i]*S/(1-S)*(1-(n[i]+1)*np.power(S,n[i])+n[i]*np.power(S,n[i]+1))/(1+(B2[i]-
    1)*S-B2[i]*np.power(S,n[i]+1));

    axes.plot(P,vv1, marker=' ', c=c)

plt.title('BET isotherm types', fontsize=18);
axes.set_xlabel('P [bar]', fontsize=18);
axes.set_ylabel('$v_v / v_v^1$', fontsize=18);
axes.legend(['Type I', 'Type II', 'Type III', 'Type IV', 'Type V'], fontsize=18);
```

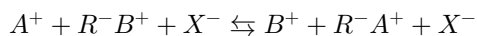


0.7 7. Equilibria in Ion Exchange Systems

In ion exchange processes adsorption takes place through by exchanging the counterions bound to charged ligands immobilised on the stationary phase. In the following examples the ligands are anionic and monovalent and are indicated with R^- .

0.7.1 7.1 Monovalent, symmetric cation exchange.

Let us consider an ion exchange system in which the cations that are selectively displaced from the liquid phase are indicated as A^+ , while the ligand counterions are indicated with B^+ . The ion exchange reaction considered can be written as:



where X^- are spectator anions.

The isotherm in the case of an ion exchange process can be derived starting from the equilibrium constant for the ion exchange reaction. If we indicate with C_A, C_B the concentration of cations A and B in the fluid phase and with n_A, n_B their respective concentration in the adsorbed phase.

The equilibrium constant is written as:

$$K_{AB} = \frac{n_A C_B}{n_B C_A} \quad (1)$$

The concentration of surface ligands is related to the concentration in the adsorbed phase of cations A^+ and B^+ by the following expression:

$$n_R = n_A + n_B \quad (2)$$

The total cationic concentration in the fluid phase C_T is analogously defined as:

$$C_T = C_A + C_B \quad (3)$$

hence the equilibrium constant becomes:

$$K_{AB} = \frac{n_A(C_T - C_A)}{(n_R - n_A)C_A} \quad (4)$$

This expression can be manipulated to provide an explicit relation between the concentration in the adsorbed phase n_A , and the concentration in the fluid phase C_A , i.e. an isotherm for the ion exchange process:

$$n_A = \frac{n_R K_{AB} C_A}{C_T + C_A(K_{AB} - 1)} \quad (5)$$

This expression can be further simplified by introducing the molar fractions in the adsorbed and fluid phases indicated with y_A and x_A , respectively:

$$y_A = \frac{K_{AB} x_A}{1 + x_A(K_{AB} - 1)} \quad (6)$$

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

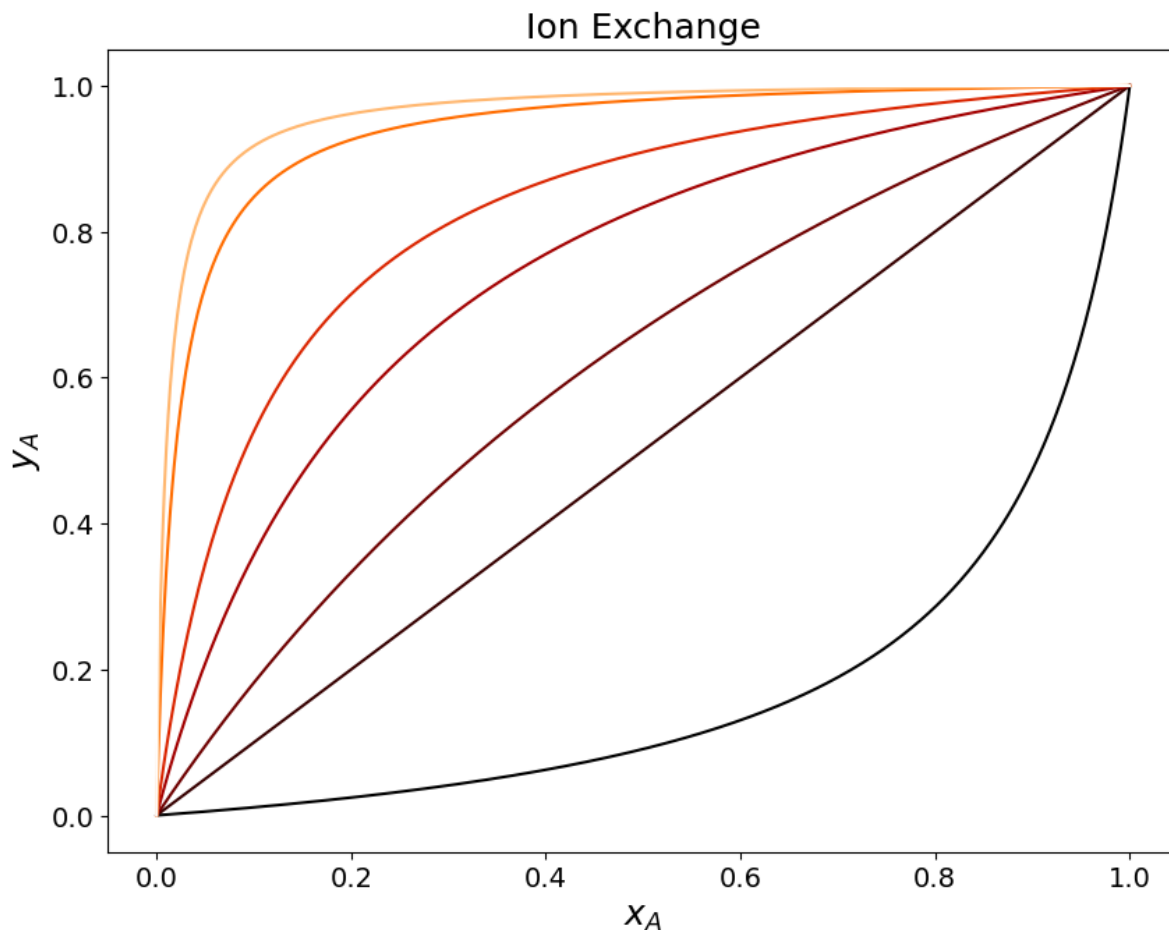
#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# molar fraction in the feed
z = 0.3
N = 500 #number of points
x = np.linspace(0, 1, N)
kAB = np.array([0.1, 1, 2, 5, 10, 50, 100, 1000]);

color=iter(cm.gist_heat(np.linspace(0,1,np.size(kAB))))

for i in range(0,np.size(kAB)):
    c=next(color)
    #Langmuir isotherm
    y = kAB[i] * x / (1 + x*(kAB[i]-1))
    axes.plot(x,y, marker=' ', c=c)

plt.title('Ion Exchange', fontsize=18);
axes.set_xlabel('$x_A$', fontsize=18);
axes.set_ylabel('$y_A$', fontsize=18);
```



The concentration of surface groups in the stationary phase n_R also plays a role in determining the amount adsorbed. In the following we report the concentration in the adsorbed phase, in equilibrium with an arbitrary molar fraction x'_A in the fluid phase as a function of the affinity towards the stationary phase (captured by the equilibrium constant K_{AB}) and of the density of surface charged ligands n_R .

```
import numpy as np
import pandas as pd
from matplotlib import cm
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Parameters:
# molar fraction in the feed
N = 100 #number of points
kAB = np.logspace(-3, 1.5, N)
nR = np.logspace(1, 3, N)
x = 0.1 ; # arbitrary value of C'
kAB, nR = np.meshgrid(kAB, nR)

nA = nR * kAB * x / (1 + x*(kAB-1))

#Plotting
figure=plt.figure(figsize=(10, 8))
axes = figure.add_subplot(projection='3d')
```

(continues on next page)

(continued from previous page)

```

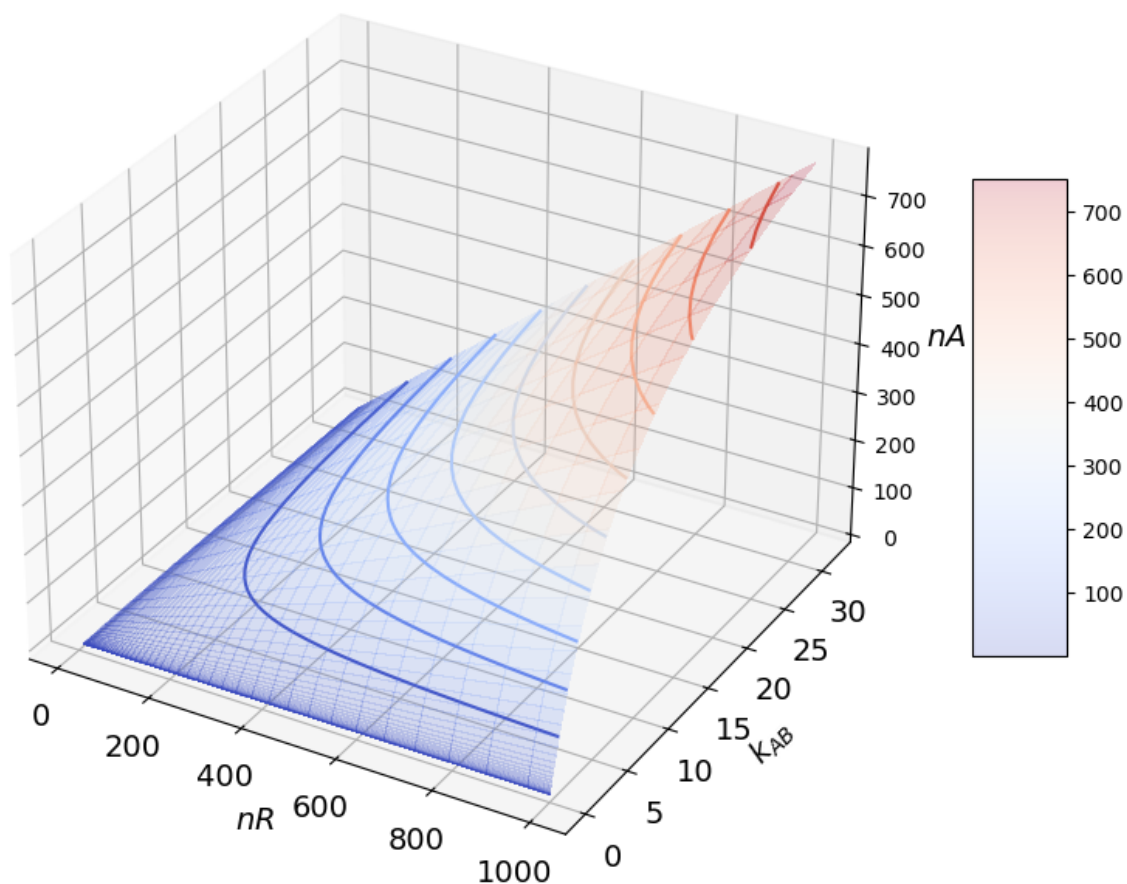
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

surf=axes.plot_surface(nR,kAB,nA,cmap=cm.coolwarm,
                      linewidth=0, antialiased=False,alpha=0.2)

contour=axes.contour(nR,kAB,nA,np.linspace(100, 800, 10),cmap=cm.coolwarm)

plt.title('Adsorbed concentration vs  $K_{AB}$  /  $nR$ ', fontsize=18);
axes.set_xlabel('$nR$', fontsize=14);
axes.set_ylabel('$k_{AB}$', fontsize=14);
axes.set_zlabel('$nA$', fontsize=14);
figure.colorbar(surf, shrink=0.5, aspect=5);

```

Adsorbed concentration vs K_{AB} / nR 

0.7.2 7.2 Example problem

An aqueous solution containing 10% by mass of NaCl and an unknown concentration of KCl exchanges cations with a stationary phase until reaching equilibrium. When equilibrated, the resin has 10.5% of sites occupied by sodium ions. The ion exchange equilibrium constant, $K_{Na+/K+}$, is equal to 10.

- Write the ion exchange reaction describing the symmetric monovalent exchange taking place in the ion-exchange column.
- Compute the molar fraction of HNO₃ in solution at equilibrium.
- If a solution with the same composition, but containing KNO₃ instead of NaNO₃ ($K_{K+/H+}=9.2$) was equilibrated with the same resin, how do you expect the fraction of occupied resin sites to change?

0.8 8. Batch Slurry Adsorption

In this notes we will use an example problem to introduce material balances in slurry adsorption processes. We will consider a Batch, process configuration and then its extension to a continuous process configuration.

0.8.1 8.1 Problem Statement

A water purification process is carried out via batch adsorption. The initial concentration of the pollutant is $0.1 [mol\ell^{-1}]$, and it needs to be lowered by at least an order of magnitude. The process is carried out in batches characterised by a water volume V_ℓ of $2 m^3$, in which $100 kg$ of adsorbent are dispersed to form a slurry. The adsorbent is characterised spherical particles with an average radius of $1 mm$ and density of $700 kg m^{-3}$. The pollutant adsorption on the surface of the stationary phase particles is well described by a linear isotherm $q = Hc^*$ with $H = 1 \times 10^{-1} [dm]$. The liquid-side mass transfer coefficient is $k_\ell = 1 \times 10^{-8} [mh^{-1}]$.

- Is it possible to carry out the requested purification with the operation parameters described in the problem statement?
- If this is the case how much time does it take to lower the pollutant concentration to the target specification?

0.8.2 8.2 Material Balance in a Batch Slurry adsorption process

Let us begin by writing the material balance for a batch membrane separator.

Since we have a batch system, no inlet or outlet streams are present and we can write:

$$V_\ell c_0 = V_\ell c + S_p q \quad (1)$$

Where V_ℓ is the volume of the liquid phase (it can be considered constant), c_0 is the initial concentration of pollutant in solution, c is the instantaneous concentration of pollutant in solution, q the instantaneous surface concentration of pollutant adsorbed on the stationary phase and S_p the total surface exposed by the stationary phase particles.

Mass transfer in these cases is dominated by bulk transport on the liquid side, thus the flux J of the pollutant leaving the fluid phase to be incorporated in the stationary phase can be written as:

$$J = -k_\ell (c - c^*) \quad (2)$$

where k_ℓ is the mass transfer coefficient on the liquid side, and c^* is the equilibrium concentration, reached at the solid/liquid interface. J has the units of number of moles per unit time per unit surface.

The differential material balance on the solute concentration can thus be written as:

$$\frac{dc}{dt} = -k_\ell \frac{S_p}{V_\ell} (c - c^*) = -k_\ell a (c - c^*) \quad (3)$$

Where a represents the particles surface area per unit of fluid volume.

The equilibrium of the adsorption process is well captured by a linear isotherm:

$$c^* = \frac{q}{H} \quad (4)$$

Using Eq. (??) and (??), Eq. (??) can be rewritten as:

$$\begin{aligned} \frac{dc}{dt} &= -k_\ell a \left(c - \frac{q}{H} \right) = -k_\ell a \left(c - \frac{(c_0 - c)V_\ell}{HS_p} \right) = \\ &= -k_\ell a c \left(1 + \frac{V_\ell}{HS_p} \right) + k_\ell a c_0 \frac{V_\ell}{HS_p} \end{aligned} \quad (5)$$

Defining: $\beta = \left(1 + \frac{V_\ell}{HS_p} \right)$, and $\alpha = \frac{V_\ell}{HS_p}$, one gets:

$$\frac{dc}{dt} + k_\ell a \beta c = k_\ell a c_0 \alpha \quad (6)$$

which is a first order, non homogeneous ODE in the form $y' + C_1 y = C_2$ with constant C_1 and C_2 coefficients.

In order to solve our problem Eq. (??) should either numerically or analytically. In this case the analytical solution can be obtained from the general solution for ODEs in the form:

$$y(t) = e^{-A(t)} \left[k_1 + \int b e^{A(t)} dt \right] \quad (7)$$

where $A(t) = \int C_1 dt$

In our case:

- $C_1 = k_\ell a \beta$, thus $A(t) = k_\ell a \beta t$
- $C_2 = k_\ell a \alpha c_0$,

thus:

$$\int b e^{A(t)} dt = \int k_\ell a \alpha c_0 e^{k_\ell a \beta t} dt = \frac{k_\ell a \alpha c_0}{k_\ell a \beta} e^{k_\ell a \beta t} = \frac{\alpha c_0}{\beta} e^{k_\ell a \beta t} \quad (8)$$

putting everything together:

$$c(t) = k_1 e^{-k_\ell a \beta t} + \frac{\alpha c_0}{\beta} \quad (9)$$

The constant C_1 can be computed from the initial conditions: at $t = 0$ $c(t) = c_0$, therefore:

$$k_1 = c_0 \left(1 - \frac{\alpha}{\beta} \right) = \frac{c_0}{\beta} \quad (10)$$

The final solution for the time-dependent concentration profile $c(t)$ is:

$$c(t) = \frac{c_0}{\beta} (e^{-k_\ell a \beta t} + \alpha) \quad (11)$$

The performance of a batch slurry process in removing the concentration in the fluid phase is limited, i.e. the concentration has an asymptotic behavior in time $\lim_{t \rightarrow \infty} c(t) = \frac{\alpha c_0}{\beta}$.

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
N = 500 #number of points
time = np.linspace(0,15, N)

#particles mass
pm=100; #kg
#particles volume
pV=100/700; #m^3

#single particle volume
radius=1E-3;
uV=(4/3)*np.pi*np.power(radius,3);
uS=4*np.pi*np.power(radius,2);

#number_of particles
NP=pV/uV;
Sp=NP*uS; #particles surface

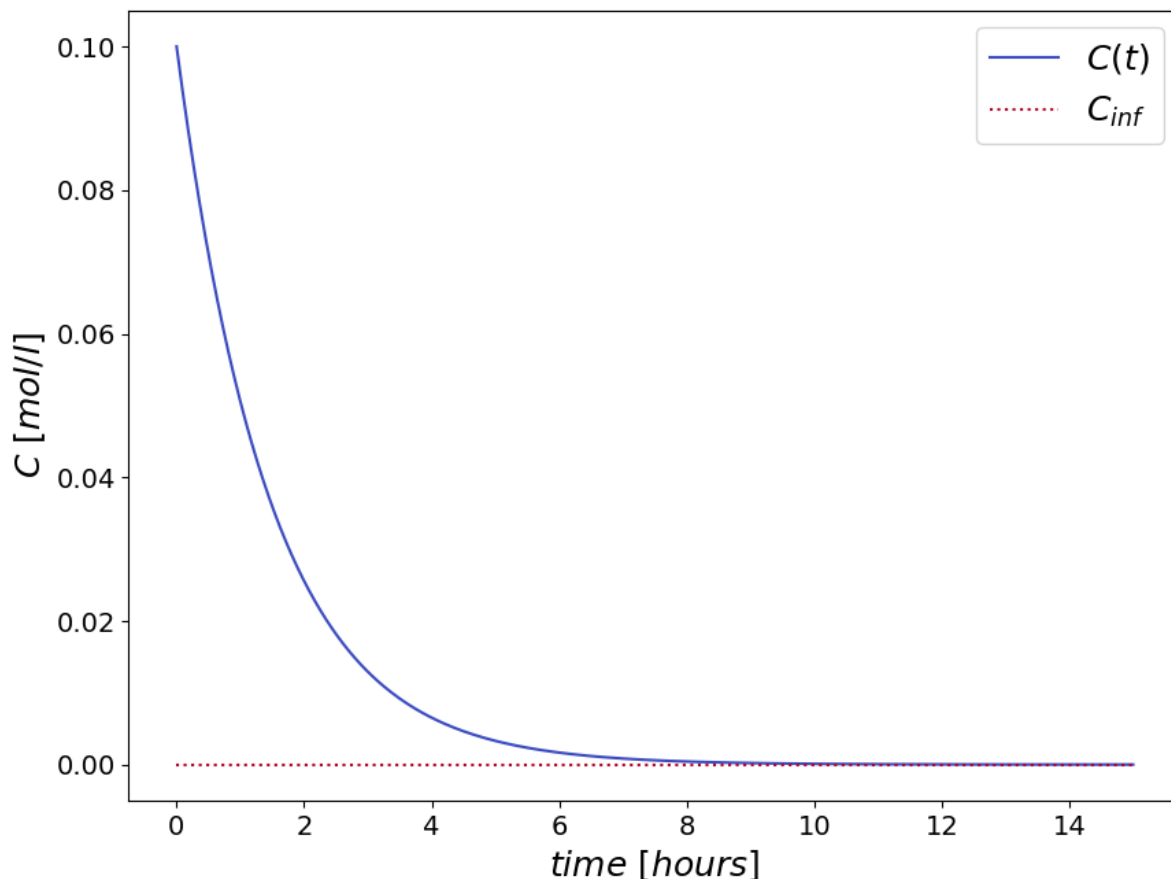
#Batch volume
Vl=2; #m^3
#Mass transfer
kl=1E-8; #m/h
#Equilibrium
H=1E-1; #

#Analytical solution 1
C0=0.1; #mol/l
alpha=Vl/H/Sp;
beta=1+alpha;
a=Sp/Vl;
C_inf= np.ones(np.size(time)) * C0*alpha/beta
C = C0/beta * (np.exp(-kl*a*beta*time)+alpha)

color=iter(cm.coolwarm(np.linspace(0,1,2)))
c=next(color)
axes.plot(time,C, marker=' ',c=c)
c=next(color)
axes.plot(time,C_inf,marker=' ',linestyle='dotted',c=c)

plt.title(' ', fontsize=18);
axes.set_xlabel('$time\,\,[hours]$', fontsize=18);
axes.set_ylabel('$C\,\,[mol / l]$', fontsize=18);
axes.legend(['$C(t)$', '$C_{inf}$'], fontsize=18);

```

0.8.3 8.3 Continuous Slurry Adsorption

The slurry adsorption process can be carried out in a continuous mode. In this mode both the adsorbent and the fluid phase are continuously fed into and removed from a stirred tank.

In a continuous process it is convenient to define a as:

$$a = S\sigma/Q \quad (12)$$

where Q is the volumetric flowrate of the liquid phase, and S is the mass flowrate of the solid adsorbent phase, and σ is the surface area per unit mass of the adsorbent.

In this context the differential material balance on the solute concentration used to solve the unsteady state batch process should be rewritten as:

$$\frac{dc}{dt} = -k_\ell a(c - c^*) \rightarrow \frac{c_{in} - c_{out}}{\tau} = k_\ell a(c_{out} - c^*) \quad (13)$$

where τ is the residence time, c_{out} the concentration in the liquid phase in the unit, c_{in} the concentration in the feed, and a the amount of surface of the stationary phase particles per unit volume of the liquid phase.

This expression should be solved with the adsorption isotherm $q = kc^*$, where q is the amount adsorbed per kg of stationary phase, c is the molar concentration in solution and k is the partition coefficient typically expressed in $[l \text{ kg}^{-1}]$, and with the global material balance at steady state, which reads:

$$c_{in}Q = qS + c_{out}Q \quad (14)$$

0.8.4 Contributions

Romain Dujardin, 2 May 2022

Salar Rahbari Namin, 8 Feb 2021

Nikita Gusev, 9 Feb 2021

0.9 9. Liquid/Solid Equilibrium in multicomponent systems: defining solubility

Let us begin by writing the solid-liquid equilibrium conditions at temperature T and pressure P , between a pure crystalline phase and a multicomponent liquid phase characterised by composition $\vec{x}$. Equilibrium is stated by the equality of chemical potentials:

$$\mu_i^s(T, P) = \mu_i^\ell(T, P, \vec{x})$$

which is equivalent to the equality of the fugacity in the two phases:

$$f_i^s(T, P) = \hat{f}_i^\ell(T, P, \vec{x})$$

where $f_i^s(T, P)$ is the fugacity of a pure crystalline solid at T and P , while $\hat{f}_i^\ell(T, P, \vec{x})$, is the fugacity of a real mixture at the same temperature and pressure, at composition $\vec{x}$.

0.9.1 Fugacity of a real solution

Focusing on the right hand of Eq. \ref{eq:fugacity}, the fugacity in the \textbf{real solution} phase can be rewritten as a function of the fugacity of the \textbf{pure liquid phase} as follows:

$$\hat{f}_i^\ell(T, P, \vec{x}) = f_i^\ell(T, P) x_i \gamma_i(T, P, \vec{x})$$

where x_i is the molar fraction that realizes the equality between fugacities, i.e. the solubility of component i , $f_i^\ell(T, P) x_i$ is the fugacity of an ideal solution at molar fraction x_i , $\gamma_i(T, P, \vec{x})$ is the activity coefficient, capturing the non-idealities of the liquid phase in equilibrium with the solid. By definition of an ideal solution $f_i^\ell(T, P)$ is the fugacity of the pure solute in its liquid phase at temperature T and pressure P .

0.9.2 Fugacity of a pure crystalline solid

Let us now focus on the left-hand side of Eq. \ref{eq:fugacity}. Here, we are only concerned with the physical properties of the pure solute. The fugacity of the pure solid phase can also be written as a function of the fugacity of the pure liquid by introducing the definition of fugacity and integrating between the liquid and solid states $\ell \rightarrow s$:

$$\int_{T, P, \ell}^{T, P, s} d\mu_i = \int_{T, P, \ell}^{T, P, s} RT d \ln f_i$$

which, integrated, leads to:

$$\Delta\mu_{i, \ell \rightarrow s} = RT \ln \left(\frac{f_i^s(T, P)}{f_i^\ell(T, P)} \right)$$

and rearranged gives:

$$f_i^s(T, P) = f_i^\ell(T, P) e^{\frac{\Delta\mu_{i, \ell \rightarrow s}}{RT}}$$

where notably $\Delta\mu_{i,\ell\rightarrow s}$, is the change in chemical potential associated with the liquid to solid transition of the *pure solid*. We note that the change in chemical potential for a pure substance corresponds to the the partial molar change in free energy $\Delta g_i(T, P)$:

$$\Delta\mu_{i,\ell\rightarrow s} = \Delta g_i^s(T, P) = \Delta h_i(T, P) - T\Delta s_i(T, P)$$

Now we express the molar enthalpy change *for the pure solute* by considering a thermodynamic cycle, that goes through the melting temperature T_f at pressure P . This enables the computation of $\Delta h_i(T, P)$ and $\Delta s_i(T, P)$ via an arbitrary but convenient path, taking advantage of the fact that all terms in the free energy expression are *state functions*.

0.9.3 Enthalpy of phase transition at T and P for pure solute

The enthalpy change associated with the solid-to-liquid transition becomes:

$$\Delta h_i(T, P) = -\Delta h_{fus}(T_f) + \int_T^{T_f} C p^\ell dT - \int_T^{T_f} C p^s dT$$

where:

- $\int_T^{T_f} C p^\ell dT$ is the enthalpy change associated with raising the temperature of the *pure solute liquid* from T to T_m
- $-\int_T^{T_f} C p^s dT$ is the enthalpy change associated with decreasing the temperature of the *pure solid* from T_m to T
- $-\Delta h_{fus}(T_f)$ is the change in enthalpy associated with the liquid to solid transition of the *pure solute* at the melting temperature

For heat capacities independent of T, the enthalpic contribution reduces to:

$$\Delta h_i(T, P) \simeq -\Delta h_{fus}(T_f) + (C p^\ell - C p^s)(T_f - T)$$

0.9.4 Entropy of phase transition at T and P for pure solute

Analogously to the enthalpy term, we can write the molar change in entropy via a the same thermodynamic cycle:

$$\Delta s_i(T, P) = -\frac{\Delta h_{fus}(T_f)}{T_f} + \int_T^{T_f} \frac{C p^\ell}{T} dT - \int_T^{T_f} \frac{C p^s}{T} dT$$

where:

- $+\int_T^{T_f} \frac{C p^\ell}{T} dT$ is the entropy change associated with raising the temperature of the *pure solute liquid* from T to T_m
- $-\int_T^{T_f} \frac{C p^s}{T} dT$ is the enthalpy change associated with decreasing the temperature of the *pure solid* from T_m to T
- $-\frac{\Delta h_{fus}(T_f)}{T_f}$ is the change in entropy associated with the liquid to the solid transition of the *pure solute* at the melting temperature

Again, for heat capacities independent of T, the entropy change reduces to:

$$\Delta s_i(T, P) \simeq -\frac{\Delta h_{fus}}{T_f} + (C p^\ell - C p^s) \ln\left(\frac{T_f}{T}\right)$$

0.9.5 Free energy of phase transition at T and P for pure solute

Defining $\Delta C p_{\ell \rightarrow s} = (C p^{\ell} - C p^s)$ we can now recover the expression for $\Delta g_i^s(T, P)$, where the only assumption is that $C p^{\ell}$ and $C p^s$ are independent from the temperature T.

$$\Delta g_i^s(T, P) = -\Delta h_{fus} \left(1 - \frac{T}{T_f}\right) + \Delta C p_{\ell \rightarrow s} \left(T_f - T - T \ln \left(\frac{T_f}{T}\right)\right)$$

0.9.6 Solubility

Inserting now this expression in Eq. \ref{eq:fugacitysolid} yields:

$$f_i^s(T, P) = f_i^{\ell}(T, P) \exp \left[\frac{-\Delta h_{fus} \left(1 - \frac{T}{T_f}\right) + \Delta C p_{\ell \rightarrow s} \left(T_f - T - T \ln \left(\frac{T_f}{T}\right)\right)}{RT} \right]$$

which then can be inserted into the equality of fugacities defining equilibrium Eq. \ref{eq:fugacity} giving:

$$f_i^{\ell}(T, P) x_i \gamma_i(T, P, \vec{x}) = f_i^{\ell}(T, P) \exp \left[\frac{-\Delta h_{fus} \left(1 - \frac{T}{T_f}\right) + \Delta C p_{\ell \rightarrow s} \left(T_f - T - T \ln \left(\frac{T_f}{T}\right)\right)}{RT} \right]$$

which can be rearranged to yield an expression of the solubility x_i where the only approximation introduced is in considering the heat capacities of both the crystal and the liquid constant in T.

$$x_i = \frac{1}{\gamma_i(T, P, \vec{x})} \exp \left[\frac{-\Delta h_{fus} \left(1 - \frac{T}{T_f}\right) + \Delta C p_{\ell \rightarrow s} \left(T_f - T - T \ln \left(\frac{T_f}{T}\right)\right)}{RT} \right]$$

In this expression, the fugacity of the pure liquid solute, $f_i^{\ell}(T, P)$ here cancels out, and the term $-\Delta h_{fus}$ is not approximating a different quantity (like the enthalpy of dissolution). It is actually introduced rigorously by the convenient choice of an appropriate thermodynamic cycle for the calculation of $\Delta \mu_{i, \ell \rightarrow s}$.

0.9.7 The Schroeder-Van Laar Equation

The Schroeder-Van Laar equation for the solubility is derived from \ref{eq:solubility} by assuming that $\Delta C p_{\ell \rightarrow s}$ is negligible.

This simplification yields:

$$\Delta g_i^s(T, P) \simeq -\Delta h_{fus} \left(1 - \frac{T}{T_f}\right)$$

Thus leading to the Schroeder-Van Laar solubility equation for x_i :

$$x_i = \frac{1}{\gamma_i(T, P, \vec{x})} \exp \left[-\frac{\Delta h_{fus}}{R} \left(\frac{1}{T} - \frac{1}{T_f} \right) \right]$$

0.10 10. MSMPR Crystallizer: population balance

Let us consider an Mixed Suspension Mixed Product removal crystallizer operating at steady state at constant temperature. In these notes we will derive the analytical expression of the steady state particle size distribution by introducing a monodimensional population balance, and we will discuss its coupling with the global material balance.

0.10.1 10.0 Material Balance

Let us begin by considering the global material balance for an MSMPR crystallizer. The operation of an ideal MSMPR crystallizer is characterised by the following set of hypotheses:

- The crystalliser operates continuously
- Operation is at steady state
- The crystalliser is perfectly mixed (both with respect to the liquid and the solid phases)
- The outlet has a composition identical in all respects to the crystallizer content
- There are no crystals in the feed
- The growth rate $G=dL/dt$ is independent from the crystal size (L)
- All crystals have the same shape
- The system is not subject to agglomeration or breakage

At steady state the mass balance for the solute precipitating in an MSMPR crystallizer can be written as:

$$Q_{IN}C_{IN} = Q_{OUT}C_{OUT} + Q_{OUT}M_T \quad (1)$$

where Q_{IN} is the volumetric inlet flowrate of the liquid phase, Q_{OUT} is the volumetric outlet flowrate of the liquid phase (mother liquor), C_{IN} is the solute density in the feed, C_{OUT} is the solute density in the outlet stream, and M_T is the mass of crystals precipitated per unit volume of liquid phase. Typically $Q_{IN} \simeq Q_{OUT} = Q$ and thus $M_T = C_{IN} - C_{OUT}$.

It should be noted that M_T is determined by the process of precipitation producing a certain particles population and can be computed from the kinetics of crystal nucleation and growth, based on the population balance discussed in the neLt section.

0.10.2 10.1 Number and number population density

Let us begin by defining $N(L)$ as the **number** of crystals of characteristic length L per unit volume. The **number population density** n , or in short number density, is defined as:

$$\lim_{\Delta L \rightarrow 0} \frac{\Delta N(L)}{\Delta L} = \frac{dN(L)}{dL} = n(L) \quad (2)$$

and represents the number of crystals of length L **per unit length**, per unit volume. The number of crystals per unit volume with dimension comprised between L_1 and L_2 can thus be computed as a definite integral of the population density:

$$N_{1,2} = \int_{L_1}^{L_2} n(L)dL \quad (3)$$

Similarly the total number of crystals per unit volume is obtained as:

$$N_T = \int_0^{\infty} n(L)dL \quad (4)$$

0.10.3 10.2 Population Balance in an MSMPR crystallizer

Let us write a steady state population balance for particles with size comprised between L_1 and L_2 .

$$\Delta NV = n_1 G_1 V \Delta t - n_2 G_2 V \Delta t - Q \bar{n} \Delta L \Delta t \quad (5)$$

where ΔNV is the change in the number of particles within the length interval $\Delta L = L_2 - L_1$ per unit volume, accumulated in time Δt . The term $n_1 G_1 V \Delta t$ captures the number of crystals that outgrow length L_1 in time Δt , where G_1 is the growth rate of particles of length L_1 , V is the total volume. Similarly, the term $n_2 G_2 V \Delta t$ takes into account particles leaving the length interval ΔL , outgrowing length L_2 . Finally the term $Q \bar{n} \Delta L \Delta t$ accounts for the removal of particles with size comprised in ΔL from the crystalliser due to the magma flowrate with volumetric flowrate Q , where $\bar{n}$ represents the average population density within the length interval ΔL .

Eq. (5) can be rearranged to obtain on the left side a discrete accumulation term, i.e. the number of particles with size comprised in δL accumulated in time Δt :

$$\frac{\Delta N}{\Delta t} = n_1 G_1 - n_2 G_2 - \frac{Q}{V} \bar{n} \Delta L \quad (6)$$

At steady state there is no accumulation and Eq. (6) can be rewritten as:

$$\frac{n_2 G_2 - n_1 G_1}{\Delta L} = -\frac{Q}{V} \bar{n} \quad (7)$$

Noting that the residence time $\tau = V/Q$, and taking the limit for $\Delta L \rightarrow 0$ we get:

$$\frac{dn}{dL} = -\frac{n}{G\tau} \quad (8)$$

Which can be solved analytically, yielding the steady state number population density $n(L)$:

$$n(L) = n_0 \left(-\frac{L}{G\tau} \right) \quad (9)$$

where n_0 is an integration constant, determined from the boundary conditions, that has the physical meaning of population density of nuclei in the crystallizer at steady state.

In this formulation of the population balance we consider particles to be born as nuclei at length $L = 0$. The nucleation rate J , i.e. the number of nuclei generated per unit time per unit volume can be written as:

$$J = \left. \frac{dN}{dt} \right|_{L=0} \quad (10)$$

by applying the chain rule we can rewrite the nucleation rate as:

$$J = \left. \frac{dN}{dL} \right|_{L=0} \left. \frac{dL}{dt} \right|_{L=0} = n_0 G \quad (11)$$

where, by definition $\left. \frac{dN}{dL} \right|_{L=0} = n_0$, and $\frac{dL}{dt} = G$. We shall note that here we are considering the growth rate to be independent from the crystal size.

This expression allows us to determine the integration constant n_0 based on characteristic rates of crystal nucleation and growth, yielding:

$$n_0 = \frac{J}{G} \quad (12)$$

0.10.4 10.3 Particle size distribution and process parameters: τ , G , J

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Data
G=1
J=1
tau=np.array([0.1, 1, 2, 5, 10, 50, 100, 500]);
N = 500 #number of points
L = np.linspace(0,100, N)

color=iter(cm.coolwarm(np.linspace(0,1,np.size(tau))))

for i in range(0,np.size(tau)):
    c=next(color)
    n= J / G * np.exp(-L/G/tau[i])
    axes.plot(L,n, marker=' ', c=c)

plt.title('n(L) vs. residence time', fontsize=18);
axes.set_xlabel('$L$', fontsize=18);
axes.set_ylabel('$n$', fontsize=18);

#####
# Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Data
G=np.array([1, 2, 5, 10, 50, 100, 500]);
J=1;
tau=5;
N = 500 #number of points
L = np.linspace(0,50, N);

color=iter(cm.coolwarm(np.linspace(0,1,np.size(G))));

for i in range(0,np.size(G)):
    c=next(color)
    n= J / G[i] * np.exp(-L/G[i]/tau)
    axes.plot(L,n, marker=' ', c=c)

plt.title('n(L) vs. growth rate', fontsize=18);
axes.set_xlabel('$L$', fontsize=18);
axes.set_ylabel('$n$', fontsize=18);

#####
```

(continues on next page)

(continued from previous page)

```
# Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Data
G=1;
J=np.array([1, 2, 5, 10, 50, 100, 500]);
tau=5;
N = 500 #number of points
L = np.linspace(0,50, N);

color=iter(cm.coolwarm(np.linspace(0,1,np.size(J))));

for i in range(0,np.size(J)):
    c=next(color)
    n= J[i] / G * np.exp(-L/G/tau)
    axes.plot(L,n, marker=' ', c=c)

plt.title('n(L) vs. nucleation rate', fontsize=18);
axes.set_xlabel('$L$', fontsize=18);
axes.set_ylabel('$n$', fontsize=18);
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[1], line 1
----> 1 import numpy as np
      2 import matplotlib.pyplot as plt
      3 from matplotlib.pyplot import cm

ModuleNotFoundError: No module named 'numpy'
```

0.10.5 10.4 Moments of the PSD in a MSMPR crystallizer

The number population density $n(L)$ can be used to derive other properties of the population of particles through the calculation of the moments of the distribution. The i^{th} order moment of the number population density is defined as:

$$m_i = \int_0^{\infty} L^i n dL \quad (13)$$

Based on this definition we note that the moments from order 0 to 3 have a well defined physical meaning:

- $m_0 = \int_0^{\infty} L^0 n dL = N_L$ represents the number of particles per unit volume of liquid phase with size smaller or equal than size L . For $L \rightarrow \infty$ N_L tends to the total number of particles per unit volume of liquid phase in the crystallizer:

$$N_T = n_0 G \tau \quad (14)$$

- $m_1 = \int_0^{\infty} L^1 n dL = \mathcal{L}$ represents the total length of particles per unit volume of liquid phase with size smaller or equal than size L . For $L \rightarrow \infty$, $\mathcal{L}$ tends to the total length of particles per unit volume of liquid phase in the crystallizer:

$$\mathcal{L}_T = n_0 (G \tau)^2 \quad (15)$$

- $m_2 = \int_0^\infty L^2 n dL \propto A$ is proportional to the total area of particles per unit volume of liquid phase with size smaller or equal than size L . For $L \rightarrow \infty$, A tends to the total area of the particles per unit volume of liquid phase contained in the crystallizer:

$$A_T = 2\beta n_0 (G\tau)^3 \quad (16)$$

where β is the surface shape factor of the particles.

- $m_3 = \int_0^\infty L^3 n dL \propto V$ is proportional to the total volume of particles with size smaller or equal than size L . For $L \rightarrow \infty$, V tends to the total volume of the particles per unit volume contained in the crystallizer:

$$V_T = 6\alpha n_0 (G\tau)^4 \quad (17)$$

where α is the volumetric shape factor of the particles.

It should be noted that the total volume of the particles per unit volume of liquid phase allows to compute the total mass of particles per unit volume of liquid phase in the system:

$$M_T = \rho_c V_T = 6\rho_c \alpha n_0 (G\tau)^4 \quad (18)$$

where ρ_c is the density of the crystalline phase.

This expression for M_T allows to couple the population balance and the global material balance in an MSMR crystallizer since $M_T = C_{in} - C_{out}$, where C_{in} and C_{out} are the inlet and outlet concentrations of solute in the liquid phase.

0.10.6 10.5 Adimensional Mass distribution and dominant crystal size

In order to obtain a general relationship between the crystallization process and an average indicator of the crystalline particles size obtained in an MSMR crystallizer it is convenient to write the cumulative mass distribution in a dimensionless form. To this aim we introduce the dimensionless length $x = L/G\tau$, where G is the growth rate and τ is the residence time.

The cumulative mass distribution can thus be written as:

$$M(x) = \frac{M}{M_T} = \frac{\int_0^x x^3 \exp(-x) dx}{\int_0^\infty x^3 \exp(-x) dx} = 1 - \left(1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3\right) \exp(-x) \quad (19)$$

The corresponding mass population density is computed differentiating $M(x)$ with respect to x :

$$m(x) = \frac{dM(x)}{dx} = \frac{1}{6}x^3 \exp(-x) \quad (20)$$

The mass-based population density $m(x)$ has a maximum for $x = 3$, hence the most probable - often referred to as the modal or dominant particle size can be computed as:

$$L_D = 3G\tau \quad (21)$$

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Data
```

(continues on next page)

(continued from previous page)

```

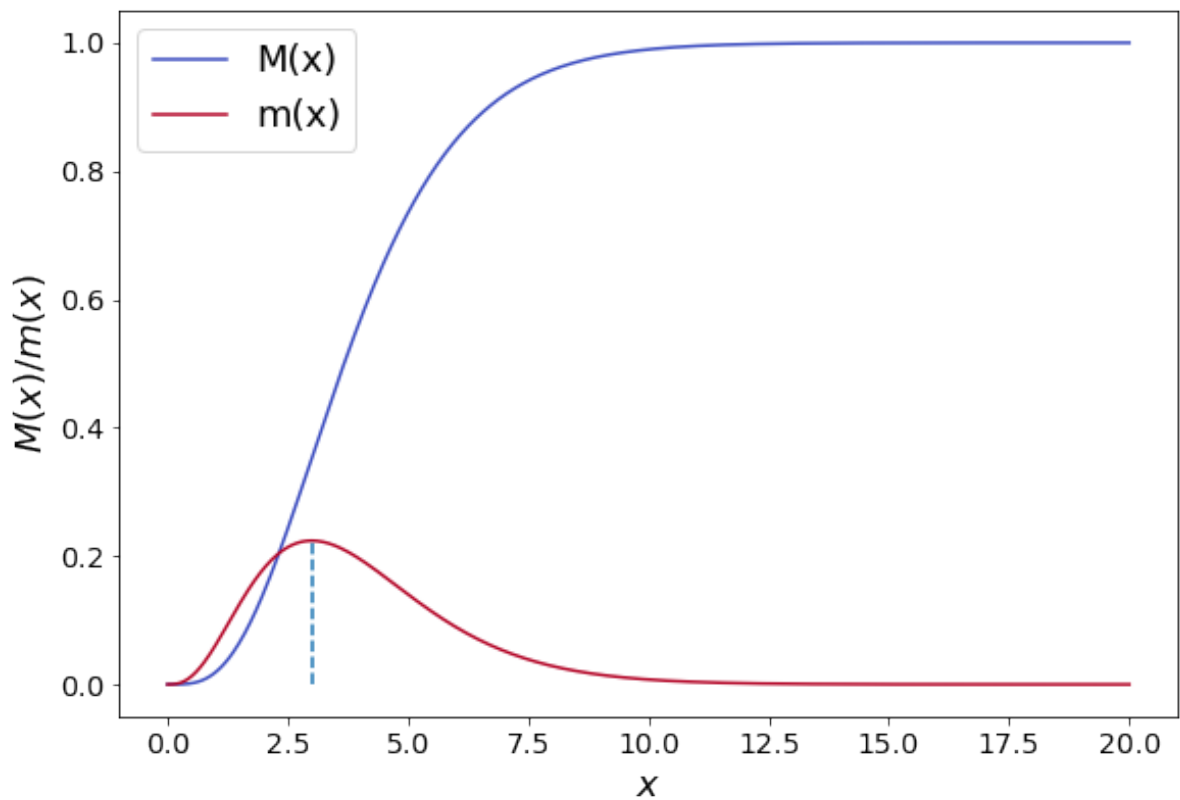
x = np.linspace(0,20, N)

color=iter(cm.coolwarm(np.linspace(0,1,2)))

c=next(color)
M=1-(1+x+0.5*np.power(x,2)+1/6*np.power(x,3))*np.exp(-x)
axes.plot(x,M, marker=' ', c=c)
c=next(color)
m=1/6*np.power(x,3)*np.exp(-x)
axes.plot(x,m, marker=' ', c=c)
axes.plot(np.array([3,3]),np.array([0, 1/6*np.power(3,3)*np.exp(-3)]), '--',)

axes.set_xlabel('$x$', fontsize=18);
axes.set_ylabel('$M(x)/m(x)$', fontsize=18);
axes.legend(['M(x)', 'm(x)'], fontsize=18);

```



0.10.7 Contributions

Guilherme Pizarro Werner, 23 February 2021

Ahmad Eblagh, 13 March 2021

0.11 Week 2 - Exercise

0.11.1 Problem Statement

A perfectly mixed gas permeation module is used to separate carbon dioxide from nitrogen using a poly (2,6-dimethylphenylene oxide) membrane ($P_{CO_2} = 0.034 \times 10^{-13}$ $P_{N_2} = 0.089 \times 10^{-14} [cm^3(STP)/cmsPa]$). The process carried out at a temperature of $25^\circ C$. The feed flowrate is 20.0 mol% carbon dioxide. The module has $15.0 m^2$ of membrane. The module is operated with a retentate pressure of 5.5 atm and a permeate pressure of 1.01 atm.

The permeate is enriched to 38.0 mol% in carbon dioxide. The membrane thickness is $\ell = 2.0 \times 10^{-4}$ cm.

- Draw a scheme of the unit clearly labelling all the streams and relevant variables.
- Completely characterise the unit, by computing feed, permeate and retentate flow rates, cut, and retentate composition.

0.11.2 Solution trace

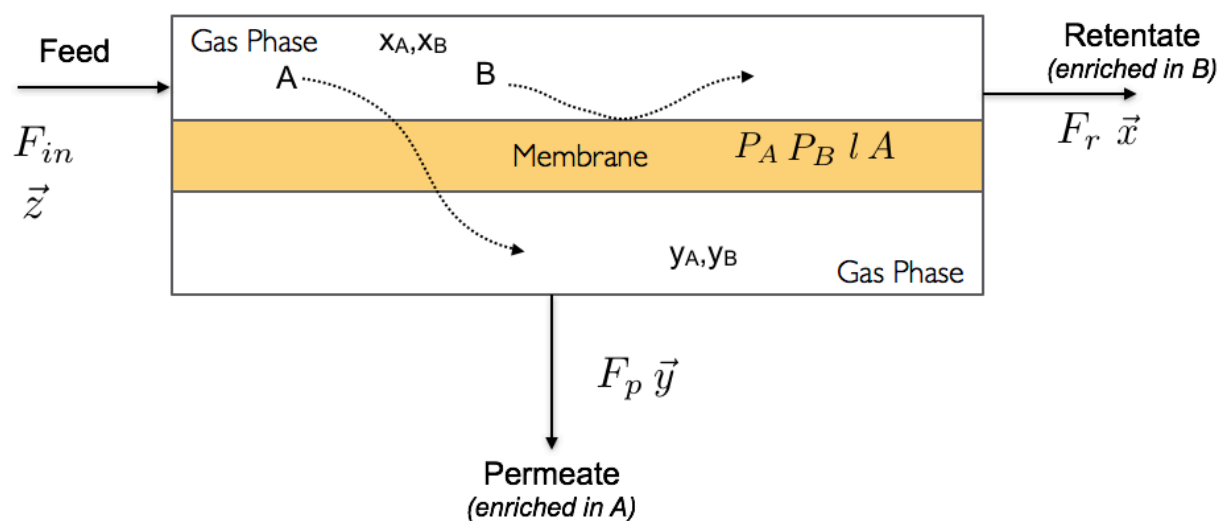


Fig. 1: Schematics of a membrane process with all variables defined.

Unknowns: $x, F_p, F_{in}, F_r, \theta$

We can start computing the ideal separation factor and the composition in the retentate:

$$\alpha = P_{CO_2} / P_{N_2}$$

and then using the rate transfer equation to computing x :

$$x = \frac{y \left[1 + \frac{p_r}{p_r} (1 - y) (\alpha_{AB} - 1) \right]}{\alpha_{AB} - (\alpha_{AB} - 1) y}$$

Then we can compute the cut:

$$\theta = \frac{z - x}{y - x}$$

and the permeate flowrate from the flux equation:

$$F_p = \frac{A P_{CO_2} \rho_{STP}}{l y} (x P_r - y P_p)$$

where ρ_{STP} is the molar density of an ideal gas at standard T and P.

Finally, the retentate flowrate can be computed as:

$$F_r = F_{in} - F_p$$

```
## Solution trace
# Variables
A=15 * 1E4 # [m^2 * cm^2 / m^2]
P_r=5.5 * 101325 # [atm] * [Pa / atm]
P_p=1.01 * 101325 # [atm] * [Pa / atm]
l=2.0*1E-4 # cm
rho_STP=1/22.4E3 # / [mol/cm3(stp)]
PCO2=0.034E-13 * 3600 # [cm3(STP) * cm/cm2/s/Pa] * [s/h]
PN2=0.089E-14 * 3600# [cm3(STP) * cm/cm2/s/Pa] * [s/h]
# Permeate Composition
y=0.38
# Feed Composition
z = 0.20

# 1. compute the retentate composition:
alpha=PCO2/PN2
x= y * (1 + P_p/P_r * (1-y) * (alpha - 1)) / (alpha - (alpha - 1) * y)

# 2. compute the cut:
theta=(z-x)/(y-x)

# 3. compute the permeate flowrate:
Fp=A*PCO2*rho_STP / y / l * (x*P_r-y*P_p)

# 4. compute the feed flowrate:
Fin=Fp/theta

# 5 compute the retentate flowrate:
Fr=Fin-Fp

print("\nRetentate composition: ", f"{x:.4}", " [-]")
print("Cut: ", f"{theta:.4}", "[-]")
print("Permeate flowrate: ", f"{Fp:.4}", " [mol/h]")
print("Feed flowrate: ", f"{Fin:.4}", " [mol/h]")
print("Retentate flowrate: ", f"{Fr:.4}", " [mol/h]\n")
```

```
Retentate composition:  0.1826  [-]
Cut:  0.08792 [-]
Permeate flowrate:  0.06784  [mol/h]
Feed flowrate:  0.7715  [mol/h]
Retentate flowrate:  0.7037  [mol/h]
```

The brute force way

```

## Solution trace
import scipy as sp
##from sp.optimize import fsolve

# Variables
A=15 * 1E4 # [m^2 * cm^2 / m^2]
P_r=5.5 * 101325 # [atm] * [Pa / atm]
P_p=1.01 * 101325 # [atm] * [Pa / atm]
l=2.0*1E-4 # cm
rho_STP=1/22.4E3 # / [mol/cm3(STP)]
PCO2=0.034E-13 * 3600 # [cm3(STP) * cm/cm2/s/Pa] * [s/h]
PN2=0.089E-14 * 3600# [cm3(STP) * cm/cm2/s/Pa] * [s/h]
# Permeate Composition
y=0.38
# Feed Composition
z = 0.20

alpha=PCO2/PN2

def equations(vars):
    x, Fin, Fp, Fr, theta = vars
    eq1 = ((theta - 1) / theta) * x + (z / theta) - y
    eq2 = x - (y * (1 + P_p/P_r * (1-y) * (alpha - 1)) / (alpha - (alpha - 1) * y))
    eq3 = Fin - Fr - Fp
    eq4 = Fp-A*PCO2*rho_STP / y / l * (x*P_r-y*P_p)
    eq5 = Fin - (Fp/theta)
    return [eq1, eq2, eq3, eq4, eq5]

x, Fin, Fp, Fr, theta = sp.optimize.fsolve(equations, (0.1, 0.1, 0.1, 0.1, 0.1))

print("\nRetentate composition: ", f"{x:.4}", " [-]")
print("Cut: ", f"{theta:.4}", " [-]")
print("Permeate flowrate: ", f"{Fp:.4}", " [mol/h]")
print("Feed flowrate: ", f"{Fin:.4}", " [mol/h]")
print("Retentate flowrate: ", f"{Fr:.4}", " [mol/h]\n")

```

```

Retentate composition:  0.1826  [-]
Cut:  0.08792 [-]
Permeate flowrate:  0.06784  [mol/h]
Feed flowrate:  0.7715  [mol/h]
Retentate flowrate:  0.7037  [mol/h]

```

0.11.3 Contributions

- Divyansh Vashishtha, 19 Feb 2024

0.12 Week 3 - Exercise

0.12.1 Problem Statement

Compute the total membrane area necessary to produce 10 l/h purified water (0.03 wt% NaCl) through a single stage Reverse Osmosis separation of seawater (water 4 wt% NaCl). The feed pressure is 100 bar and the composition in the retentate stream is 7 wt%.

The permeate stream is at ambient pressure (1 bar), all streams are at 25° C, and the osmotic pressure in bar is captured by the semiempirical expression:

$$\pi = \frac{1.12}{14.5} T \sum_{i=1}^N c_i$$

Where T is the temperature in K, N is the total number of ionic species in solution, and ci is the concentration expressed in [mol/l].

The asymmetric membrane used for the separation has a permeance of $10^{-5} [mh^{-1}bar^{-1}]$.

Once you have solved the problem consider the following questions:

- Can you evaluate the salt rejection in this process, and salt passage?
- How would you modify the operation of the system to increase the productivity of the process?

0.12.2 Solution trace

Data:

NaCl weigh percentage: $w_{NaCl,P} = 0.03$ $w_{NaCl,F} = 4$ $w_{NaCl,R} = 7$

Water density: $\rho_{H_2O} = 1 \times 10^3 [g/l]$

Working expression:

$$c_{NaCl,i} = \frac{w_{NaCl,i}}{100} \times \frac{\rho_{Solution,i}}{MM_{NaCl}}$$

Where: *i* indicates the stream we are referring to, $\rho_{Solution,i}$ is the mass density of the NaCl solution in each stream, and MM_{NaCl} is the molar mass of NaCl. The mass density of the solution $\rho_{Solution}$ can be computed known the NaCl weight percentage and the density of water as:

$$\rho_{solution} = \frac{\rho_{H_2O}}{1 - \frac{w_{NaCl,i}}{100}}$$

Computing the Osmotic pressure difference across the membrane The expression provided in the exercise assignment allows to compute the osmotic pressure associated with an assigned concentration of NaCl in a solution of assigned NaCl concentration. *Note that the concentration appearing in such expression is the total ionic concentration, which can be computed from the NaCl concentration knowing the unit formula of NaCl.*

$$c_{TOT,i} = c_{Na^+,i} + c_{Cl^-,i} = 2c_{NaCl,i}$$

Note that in this case it is typically reasonable to assume the concentration on the feed/retentate side of the membrane to be the average between concentrations in the feed and retentate.

$$c_{TOT,F/R} = 2 \left(\frac{c_{NaCl,F} + c_{NaCl,R}}{2} \right)$$

Alternatively one can consider the feed/retentate side to be perfectly mixed and its composition constant and equal to the retentate one.

The difference in the osmotic pressure can thus be computed as:

$$\Delta\pi = \frac{1.12}{14.5} T (c_{TOT,F/R} - c_{TOT,P})$$

Computing the membrane area The flow rate through the membrane is computed by the expression:

$$F_{p,wat} = J \times A = A P_{wat} (\Delta P - \Delta\pi)$$

hence:

$$A = \frac{F_{p,wat}}{P_{wat} (\Delta P - \Delta\pi)}$$

Salt rejection Given the concentrations computed earlier salt rejection can be computed from its definition:

$$R_s = 1 - \frac{c_{NaCl,P}}{c_{NaCl,R}}$$

Salt passage is given by:

$$P_s = \frac{c_{NaCl,P}}{c_{NaCl,R}}$$

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

#
PM=1E-5; #[m/h/bar]
MMNaCl=58.5; #[g/mol]

#Feed Retentate Permeate
wtp=np.array([4, 7, 0.03])/100; #mass fractions
water_density=1E3; #[g/l]

c=wtp*water_density/(MMNaCl*(1-wtp)); #[mol/l]

C=1.12/14.5;
T=298;
pi_P=2*C*T*c[2];
pi_R=2*C*T*np.mean(c[0:2]);
Dpi=pi_R-pi_P; # [bar]

DP=100-1; # [bar]
Fpwat=10E-3; # [m3/h]

A=Fpwat/PM/(DP-Dpi)

SP=c[2]/c[1]
SR=(1-SP)

print("\nPermeate concentration: ", f"{c[2]:.4}", " [mol/l]")
print("Feed concentration: ", f"{c[0]:.4}", " [mol/l]")
print("Retentate concentration: ", f"{c[1]:.4}", " [mol/l]")

print("\nOsmotic pressure: ", f"{Dpi:.4}", " [bar]")

print("\nMembrane area: ", f"{A:.4}", " [m^2]")
print("Salt rejection: ", f"{SR:.4}", " [-]")
print("Salt passage: ", f"{SP:.2}", " [-]")
```

```
Permeate concentration: 0.00513 [mol/l]
Feed concentration: 0.7123 [mol/l]
Retentate concentration: 1.287 [mol/l]

Osmotic pressure: 45.77 [bar]

Membrane area: 18.79 [m^2]
Salt rejection: 0.996 [-]
Salt passage: 0.004 [-]
```

0.12.3 Contributions

Debugged by Nikita Gusev, 29 Jan 2021

0.13 Week 4 - Exercise

0.13.1 Problem Statement

The downstream processing of an aqueous stream containing therapeutic proteins produced in a bioreactor is carried out through an ultrafiltration membrane process. The volumetric flow to be processed is $200 \text{ m}^3 \text{ day}^{-1}$, with a suspended protein concentration of 0.5 kg m^{-3} . In order to be used in subsequent formulation stages the protein stream needs to be concentrated to 20 kg m^{-3} .

Tubular membrane modules with an exchange surface of 5 m^2 are used to carry out the ultrafiltration process.

Due to fouling, the water flow through these membranes decreases when protein concentration (C) increases following the empirical law:

$$J(C) = 3.8 \times 10^{-2} \ln\left(\frac{145}{C}\right) [\text{m h}^{-1}]$$

0.13.2 Requests

- Based on steady state calculations compute the number of membrane modules necessary for the operation of this process in a single-stage feed-and-bleed configuration.
- Consider now a cascade of two stages. Compute the optimal number of membrane modules and their arrangement in stages in order to achieve the specifics required. Compute all the concentrations and volumetric flows in your process configuration of choice. Define clearly your chosen criterion for optimality.

0.13.3 Solution:

[contributed by Max Shone, February 2024]

0.13.4 Request 1

To compute the number of membrane modules necessary for the operation of this process in a single-stage feed-and-bleed configuration, a mass balance and volume balance can be set up:

Mass Balance:

$$Q_{in} \times C_{in} = Q_{out} \times C_{out}$$

Volume Balance:

$$Q_{in} = Q_{out} + J(C)A$$

To solve this problem, the following steps are taken:

- Solve mass balance for Q_{in} .
- Solve empirical formula for flux through membrane.
- Rearrange volume balance for area and solve, rounding the final answer up.

```
import numpy as np
Qin = 200/24 #m^3/hr
Cin = 0.5 #kg/m^3
Cout = 20 #kg/m^3

#equations

#mass balance
Qout = Qin*Cin/Cout

#Finding flux going through membrane
Jc = (3.8*10**(-2)*np.log(145/Cout))

#Volume balance rearranged for A
A = (Qin-Qout)/Jc
n=np.ceil(A/5)
print("\nFlow rate out: ", f"{Qout:.4}", " [m^3/h]")
print("\nFlux: ", f"{Jc:.4}", " [m/h]")
print("\nMembrane Area: ", f"{A:.4}", " [m^2]")
print("\nNumber of stages: ", f"{n}", " [-]")
```

```
Flow rate out:  0.2083  [m^3/h]
```

```
Flux:  0.07528  [m/h]
```

```
Membrane Area:  107.9  [m^2]
```

```
Number of stages:  22.0  [-]
```

0.13.5 Request 2

To find the optimal membrane configuration for a cascade of two stages, we would like to find the least number of total membranes that will result in an outlet concentration of 20 kg m^{-3} .

This can be done through completing a mass and volume balance over the first membrane, assuming the outlet concentration (C_1) value, followed by completing the same calculations over the second membrane, with an outlet concentration of 20 kg m^{-3} . The following graph shows all possible values of C_1 between 0.5 and 20 kg m^{-3} .

```
import numpy as np
import matplotlib.pyplot as plt

#Question 2 Open exercise
#Initialising variables
Q0 = 200/24; #[m^3/day] * [day/hour]
C0 = 0.5; #kg/m^3
C1 = np.linspace(0.5, 20, num=400) #kg/m^3
C2 = 20; #kg/m^3
JC = 3.8*10**(-2)*np.log(145./C1)

#Membrane Surface Area
MSA = 5

#Calculations over first membrane
Q1 = Q0*C0/C1
A1 = (Q0-Q1)/JC
n1 = np.ceil(A1/MSA)

#Calculations over second membrane
Q2 = Q1*C1/C2
A2 = (Q1-Q2)/JC[-1]
n2 = np.ceil(A2/MSA)

#Total membranes
n = n1+n2

# Plotting graphs
plt.plot(C1, n, label="Total membranes")
plt.plot(C1, n1, label="Membrane 1")
plt.plot(C1, n2, label="Membrane 2")

# Adding legend, labels, and title
plt.legend()
plt.xlabel("C1")
plt.ylabel("Number of Membranes")
plt.title("Number of Membranes vs C1")

# Show plot
plt.show()

#Finding index of minimum number of membranes
minN_index = np.argmin(n)

#Finding values at minimum number of membranes
minN = n[minN_index]
minN1 = n1[minN_index]
minN2 = n2[minN_index]
```

(continues on next page)

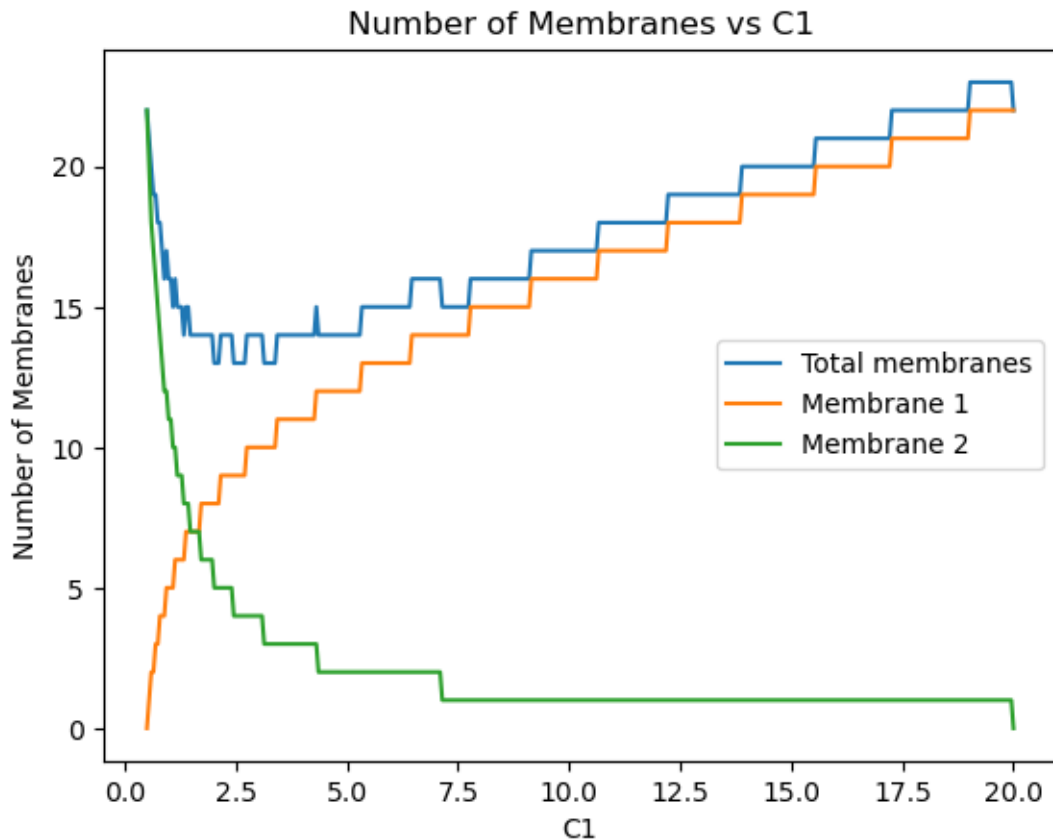
(continued from previous page)

```

minC1 = C1[minN_index]

print("\nStage 1 outlet concentration: ", f"{minC1:.4}", " [kg/m^3]")
print("\nMinimum number of membranes for stage 1: ", f"{minN1}", " [-]")
print("\nMinimum number of membranes for stage 2: ", f"{minN2}", " [-]")
print("\nMinimum number of total membranes: ", f"{minN}", " [-]")

```



```

Stage 1 outlet concentration:  2.015  [kg/m^3]

Minimum number of membranes for stage 1:  8.0  [-]

Minimum number of membranes for stage 2:  5.0  [-]

Minimum number of total membranes:  13.0  [-]

```

This can also be solved with GAMS:

#Note: This code cannot be run in Jupyter notebook, copy paste into GAMS.

```

Parameters
Q0,C0,C2;
Q0=200/24;
C0=0.5;
C2=20;

```

```

Variables

```

(continues on next page)

(continued from previous page)

```
n, Q1, C1, Q2, A1, A2
;

Equations
mb1, mb2, area, vb1, vb2
;

mb1.. Q0=e=(3.8*10**(-2))*log(145/C1)*A1+Q1;
vb1.. Q1=e=Q0*C0/C1;
mb2.. Q1=e=(3.8*10**(-2))*log(145/20)*A2+Q2;
vb2.. Q2=e=Q1*C1/C2;
area.. n=e=A1/5+A2/5;

Model process /all/;
c1.l=5;
Solve process using NLP minimising n;
```

0.13.6 Refinement of the GAMS Solution:

[Contributed by Divyansh Vashishtha, March 2024]

In this version only an integer number of membrane modules can be adopted in every stage. In this version, only an integer number of membrane modules can be adopted in every stage.

```
Parameters
Q0 initial volumetric flowrate coming into stage 1 [m^3 h^-1],
C0 initial protein concentration coming into stage 1 [kg m^-3 ],C2 final protein_
    concentration leaving stage 2 [kg m^-3];
*Initialisation of Parameters from the Process Description: Q0, C0, C2.
Q0=200/24;
C0=0.5;
C2=20;

Variables
n total number of membrane modules across the two stages [decimal],
Q1 volumetric flowrate leaving the first stage [m^3 h^-1],
C1 concentration of protein leaving the first stage [kg m^-3],
Q2 volumetric flowrate leaving the second stage [m^3 h^-1],
A1 Total area of membranes required in stage 1 [m^2],
A2 Total area of membranes required in stage 2 [m^2],
n1 number of membrane modules required for stage 1 [decimal],
n1_rounded minimum number of membrane modules required for stage 1 [integer],
n2 number of membrane modules required for stage 2 [decimal],
n2_rounded minimum number of membrane modules required for stage 2 [integer],
nx rounded up value of the total membranes required [integer].
;

Equations
mb1 equation for the mass balane of the solute over stage 1,
mb2 equation for the mass balance of the solute over stage 2,
area1 equation for the area of membrane required over the first stage,
vb1 equation for the volume balance over stage 1,
vb2 equation for the volume balance over stage 2,
area2 equation for the area of membrane required over the second stage,
roundn1 equation that rounds up the decimal value of the number of membrane modules_
```

(continues on next page)

(continued from previous page)

```

↳obtained in the first stage ,
roundn2 equation that rounds up the decimal value of the number of membrane modules.
↳obtained in the second stage,
total_n equation that determines the total number of membrane modules obtained over
↳the two stages [returns a decimal],
roundN equation that displays the rounded number of total membrane modules over the
↳two stages;

vb1.. Q0=e=(3.8*10**(-2))*log(145/C1)*A1+Q1;
mb1.. Q1=e=Q0*C0/C1;
vb2.. Q1=e=(3.8*10**(-2))*log(145/C2)*A2+Q2;
mb2.. Q2=e=Q1*C1/C2;
area1.. n1 =e= A1/5;
area2.. n2 =e= A2/5;
roundn1.. n1_rounded =e= ceil(n1);
roundn2.. n2_rounded =e= ceil(n2);
total_n.. n =e= n1 + n2;
roundN.. nx =e= ceil(n);

Model process /all/;
c1.l=5;
Solve process using DNLP minimising n;
display n.l, n1_rounded.l, n2_rounded.l, nx.l;

```

0.14 Week 5 - Exercise

0.14.1 Problem Statement

A protein mixture has been separated through an ion-exchange chromatography process. The residence time of the column in non-adsorbing conditons is $t_0 = 1$ min. The chromatogram resulting from the separation is reported in the following:

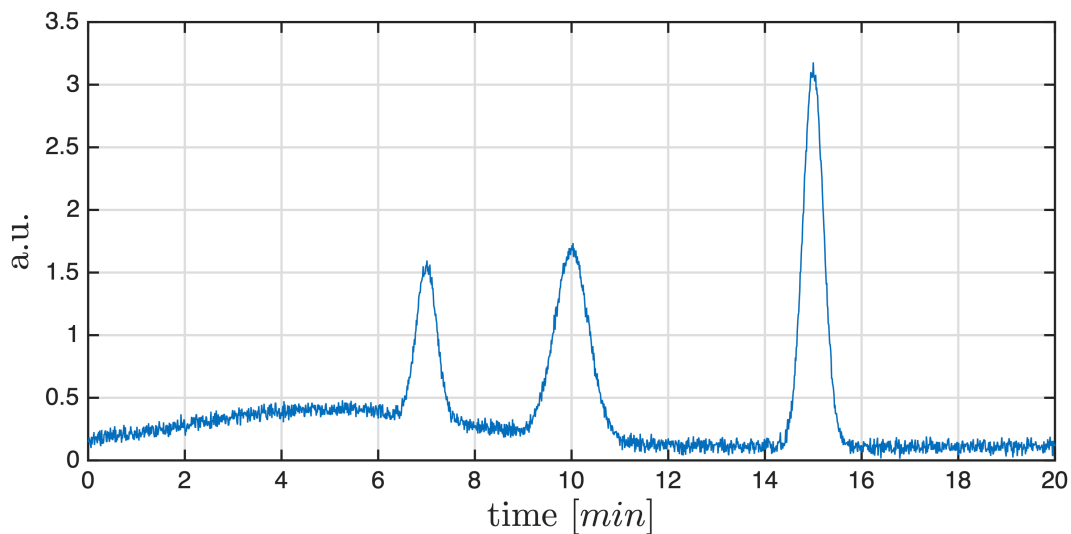


Fig. 1: Experimental chromatogram.

Determine:

- How many components in the mixture can you identify from the chromatogram?
- What is the selectivity of this column for each couple of components?
- Assuming a linear adsorption isotherm holds in all cases, what conclusions can you draw about adsorption thermodynamics of each component in the column?
- What is the number of theoretical plates associated to the elution of each component you have identified? If the column is 10 cm long, what is the height equivalent of a theoretical plate for each component of the mixture?
- Compute the resolution relative to each couple of components in the chromatogram. Does it depend only on the adsorption thermodynamics?
- Comment on the separation performances of the column. Does it allow for a sharp resolution of all components?

Solution

1. The chromatogram clearly indicates three components, let us indicate them with A, B, and C. The retention times for these components are:

- $t_{r,A} = 7$
- $t_{r,B} = 10$
- $t_{r,C} = 15$

2. The selectivity can be computed for every pair of components as:

$$S_{i,j} = \frac{t_{r,i} - t_0}{t_{r,j} - t_0}$$

```
import numpy as np

tr=np.array([7,10,15]);
t0=1
SEL=np.zeros((np.size(tr),np.size(tr)));

for i in range(0,np.size(tr)):
    for j in range(0,np.size(tr)):
        SEL[i,j]=(tr[i]-t0)/(tr[j]-t0)

print('Selectivity i,j\n',SEL)
```

```
Selectivity i,j
[[1.          0.66666667  0.42857143]
 [1.5         1.          0.64285714]
 [2.33333333  1.55555556  1.          ]]
```

3. Since $S_{i,j} = H_i/H_j$ we can determine the relative affinity of components A, B and C for the stationary phase. Component C shows the highest affinity for the stationary phase, followed by B and then A.

4/5. The number of theoretical plates associated to the elution of components A, B, and C can be obtained applying the relationship $N = 16 * (tr/tw)^2$. The resolution for every couple of components is computed as: $R_{i,j} = \frac{t_{r,i} - t_{r,j}}{\frac{1}{2}(t_{w,i} + t_{w,j})}$

```
import numpy as np

tr=np.array([7,10,15]);
```

(continues on next page)

(continued from previous page)

```

tw=np.array([1,2,1.5]);
L=10 #cm

N=(16*(tr/tw)**2)

H=L/N

R=np.zeros((np.size(tr),np.size(tr)));

for i in range(0,np.size(tr)):
    for j in range(0,np.size(tr)):
        R[i,j]=abs((tr[i]-tr[j])/(2*(tw[i]+tw[j])))

print('\nHeight of a theoretical plates ',H,' [-]')
print('Number of theoretical plates ',H,' cm')
print('Resolution i,j\n',R)

```

```

Height of a theoretical plates [0.0127551 0.025      0.00625 ] [-]
Number of theoretical plates [0.0127551 0.025      0.00625 ] cm
Resolution i,j
[[0.         0.5         1.6         ]
 [0.5        0.         0.71428571]
 [1.6        0.71428571 0.         ]]

```

0.15 Week 6 - Exercise

0.15.1 Problem Statement

Continuous slurry adsorption is carried out to reduce the pollutant content in an aqueous stream from an inlet concentration of $1 \times 10^{-2} \text{ mol l}^{-1}$. The flowrate of the aqueous stream is $50 \text{ m}^3 \text{ h}^{-1}$. The adsorbent flowrate is 750 kg h^{-1} . The residence time is 3 h.

Adsorption is controlled by external mass transfer with a coefficient $k_L = 0.5 \text{ m h}^{-1}$. The adsorbent particles have a surface area of $10 \text{ m}^2 \text{ kg}^{-1}$ and a density of 1850 kg m^{-3} . Consider a linear isotherm of the form $q = kc$, where $q [\text{mol kg}^{-1}]$ is the amount of phenol adsorbed per kg of adsorbent, c is the molar concentration of pollutant in solution and $k = 1 \times 10^4 [\text{l kg}^{-1}]$ is the partition coefficient.

Determine the steady state concentration of pollutant in the aqueous stream leaving the unit.

0.16 Week 8 - Exercise

0.16.1 Problem Statement 1

- In a system with two monotropic polymorphs, is it true that the polymorph with the lowest solubility is thermodynamically more stable?
- Can you demonstrate it using the definition of solubility?

0.16.2 Solution Trace

[contributed by Nikita Gusev, 9/March/2021]

- Crystal polymorphs are defined as substances that are chemically identical but exist in more than one crystal form. A polymorphic transition is a reversible transition of a solid crystalline phase at a certain temperature and pressure (the inversion point) to another phase of the same chemical composition with a different crystal structure. For a monotropic system, plots of the solubility of the various polymorphs against temperature do not cross, meaning that different polymorphs always have different solubilities, and different stability. Solution of a crystal is favored if the energy of dissolution is greater than the lattice energy, i.e., by dissolving, a compound forms a more thermodynamically stable state. In that sense, if the solubility of a polymorph is greater, means that this polymorph is less stable in its crystalline state, and vice versa, the polymorph with the lowest solubility is thermodynamically more stable in its crystalline state.
- The chemical potential of a solid crystalline phase is given by:

$$\mu_i^s \simeq \mu_0 + RT \ln x_i^*$$

where x_i^* is the solubility.

From the equation above it can be seen than the lower is the solubility, the lower is the chemical potential and we know that the phase with the lowest chemical potential at any temperature is the most stable phase at that temperature. This again shows that a less soluble polymorph is more stable.

0.16.3 Problem Statement 2

An aqueous feed of 10000 kg h^{-1} , saturated with BaCl_2 at 100°C , enters a crystallizer that can be simulated with the MSMPR model. The slurry leaves the crystallizer at 20°C with crystals of the dihydrate. The crystallizer has a volume of 2.0 m^3 . From laboratory experiments, the crystal growth rate is essentially constant at $4.0 \times 10^{-7} \text{ m/s}$. The solubility of BaCl_2 is $58.3 \text{ g/100 g water}$ at 100°C and $35.7 \text{ g/100 g water}$ at 20°C . The molar mass of BaCl_2 is 208.27 g/mol , the the dihydrate, $\text{BaCl}_2 \cdot 2\text{H}_2\text{O}$ has a molar mass of 244.31 g/mol . The density of the crystals is 3097 kg/m^3 , the density of the solution in the crystallizer is 1279 kg/m^3

- Determine the mass flow rate of crystals in the product stream.
- Compute the predominant crystal size L_D in mm.

0.16.4 Solution Trace

[Nikita Gusev and Guilherme Pizarro Werner, 9/March/2021]

A total material balance around the crystallizer (assuming steady state):

$$F = V + L + S$$

where:

- F = total feed rate (kg/h)
- V = evaporation rate
- L = Liquid phase flow rate kg/h of liquid
- S = Crystals flow rate kg/h of crystals

Assuming evaporation is negligible and rearranging for L :

$$L = F - S$$

Material balance for $BaCl_2$:

$$x_F F = x_L L + x_S S$$

- where x_F, x_L, x_S are the mass fractions of $BaCl_2$ in feed, liquid and crystals, respectively.

F is known, and mass fractions can be found from the data, so we have two equations and two unknowns (L and S), and so there is 0 degrees of freedom and the problem can be solved.

Solving simultaneously using the expressions above:

$$x_F F = x_L (F - S) + x_S S$$

Rearranging:

$$S = F \frac{x_F - x_L}{x_S - x_L}$$

Now finding the mass fractions:

$$x_{F,L} = \frac{w_{F,L}}{w_{F,L} + 100}$$

where $w_{F,L}$ is the solubility mass of solute per 100 g of water.

$$x_S = \frac{MM_{BaCl_2}}{MM_{BaCl_2} + MM_{H_2O}}$$

```
## Solution trace
# Variables
wF=58.3 #g/100 g water
wL=35.7 #g/100 g water
MMBaCl2= 208.27 #g/mol
MMBaCl22H2O=244.31 #g/mol
F=1E4 #kg/h

xF=wF/(wF+100);
xL=wL/(wL+100);
xS=MMBaCl2/MMBaCl22H2O;

S=F*(xF-xL)/(xS-xL)

L=F-S

print("\nFeed mass fraction ", f"{xF:.4}", " [-]")
print("Liquid phase mass fraction ", f"{xL:.4}", " [-]")
print("Crystal mass fraction ", f"{xS:.4}", " [-]")
print("Crystals flowrate: ", f"{S:.4}", " [kg/h]")
print("Liquid phase flowrate: ", f"{L:.4}", " [kg/h]")
```

```
Feed mass fraction  0.3683  [-]
Liquid phase mass fraction  0.2631 [-]
Crystal mass fraction  0.8525 [-]
Crystals flowrate:  1.785e+03  [kg/h]
Liquid phase flowrate:  8.215e+03  [kg/h]
```

The modal or dominant particle size can be computed as:

$$L = 3G\tau$$

where:

- τ = average residence time
- G = growth rate

We need to find the average residence time:

$$\tau = \frac{V}{Q}$$

where:

- V = volume of the crystallizer
- Q = outlet volumetric flowrate

But first we need to calculate the outlet volumetric flowrate:

$$Q = \frac{S}{\rho_S} + \frac{L}{\rho_L}$$

```
## Solution trace
# Variables
rho_C=3097
rho_L=1290
V=2 #m3
G=4E-7 #m/s

Q=L/rho_L+S/rho_C #m3/h

tau=V/Q*3600 #s

LD=3*G*tau*1E3 #mm

print("\nThe volumetric flowrate ", f"{Q:.4}", "[m3/h]")
print("The residence time ", f"{tau:.4}", "[s]")
print("The predominant crystal size is ", f"{LD:.4}", "[mm]")
```

```
The volumetric flowrate  6.945 [m3/h]
The residence time  1.037e+03 [s]
The predominant crystal size is  1.244 [mm]
```

0.17 Week 9 - Exercise

0.17.1 Problem Statement

The cumulative particle size distribution $N(L)$ has been measured for two different MSMPR crystallization processes (A and B) and is reported in Fig. 1. Both processes are producing the same compact crystals, under different residence times and temperatures. The growth rate is known to be independent from both temperature and composition ($G = \text{const}$). The nucleation rate J instead depends exponentially on the inverse of the temperature through an Arrhenius-like expression ($J = A \exp -B/T$, where A and B are constants).

Tasks:

- Compute and plot the number density, length, area, and normalised mass distribution for these two processes. Download the dataset: [plain text](#), [excel xlsx format](#).
- Compute the dominant length L_D obtained from both processes.
- Can you say anything about the residence time in the two crystallisers?
- Which process is operating at the highest temperature?

Solution

- Contributed by Guilherme Pizarro Werner
- Contributed by Nikita Gusev

0.18 Week 10 - Exercise**0.18.1 Problem Statement**

Consider a drying process evolving through two stages of *i* constant drying rate, and *ii* falling rate (first order kinetics).

During stage *i* the drying rate *per unit area* is constant, and indicated with $R_c = m \times f_c = \text{const.}$

Where:

- m is a drying rate constant
- $f_c = w_c - w_e$
- w_c is the critical moisture content
- w_e is the equilibrium moisture content
- w_0 is the initial moisture content

During stage *ii* the rate of drying is proportional to the free moisture content ($f = w - w_e$). The falling rate regime stops at the equilibrium moisture content w_e .

0.18.2 Tasks

1. Derive an expression for the time necessary to reduce the moisture content from w_0 to w_c .
2. Derive an expression for the time necessary to reduce the moisture content from w_c to w_e in the (linear) falling regime.
3. Derive an expression for the time necessary to reduce the moisture content from w_0 to $w_e < w < w_c$.
4. If a system, characterised by critical moisture content of 18% and an equilibrium moisture content of 2% takes 7300 seconds to reach a final moisture content of 5% from an initial moisture content of 20%, how long will it take, in the same conditions, to dry a system from 30% to 3%.
5. For the same system produce a plot of the drying time as a function of the kinetic constant m .

0.18.3 Solution

Task 1.

In order to derive an expression for the time necessary to obtain a reduction in the moisture content from an initial value w_0 to the critical moisture content w_c we shall start by realising that for $w \geq w_c$ the rate of drying (per unit area) $R = \text{const} = R_c = m(w_c - w_e)$.

Hence we can write the rate of reduction in moisture content $\frac{dw}{dt}$ as:

$$\frac{dw}{dt} = -R_c A$$

which can be solved by separating the variables

$$-\frac{dw}{R_c A} = dt$$

and integrating:

$$-\frac{1}{AR_c} \int_{w_0}^{w_c} dw = \int_0^{t_c} dt$$

leads to:

$$t_c = \frac{w_0 - w_c}{AR_c} = \frac{w_0 - w_c}{mA(w_c - w_e)}$$

Task 2.

In order to derive an expression for the time necessary to obtain a reduction in the moisture content from the critical value w_c to the equilibrium moisture content let us begin first by deriving an expression for a generic $w_e \geq w < w_c$. In this case we know that the rate is linearly dependent on w according to the expression given in the text $R = m(w - w_e)$. The rate of reduction in moisture content $\frac{dw}{dt}$ now becomes:

$$\frac{dw}{dt} = -mA(w - w_e)$$

which can be solved again by separating the variables

$$-\frac{1}{mA} \frac{dw}{w - w_e} = dt$$

and integrating:

$$-\frac{1}{mA} \int_{w_c}^{w'} \frac{dw}{w - w_e} = \int_0^{t_w} dt$$

leads to:

$$t_w = \frac{1}{mA} \ln \left(\frac{w_c - w_e}{w - w_e} \right)$$

we note that since $w - w_e$ is at the denominator,

$$\lim_{w \rightarrow w_e} \frac{1}{mA} \ln \left(\frac{w_c - w_e}{w - w_e} \right) = +\infty$$

so for the moisture content approaching its equilibrium value the time necessary diverges.

Task 3.

The total time necessary to dry from an initial moisture content $w_0 > w_c$ to a final moisture content $w < w_c$ is equal to:

$$t_{tot} = \frac{1}{mA} \left(\frac{w_0 - w_c}{w_c - w_e} + \ln \frac{w_c - w_e}{w - w_e} \right)$$

Task 4.

Using the expression derived earlier one can compute first

$$mA = \frac{1}{t_{tot,1}} \left(\frac{w_{0,1} - w_c}{w_c - w_e} + \ln \frac{w_c - w_e}{w_1 - w_e} \right)$$

and then once mA is known,

$$t_{tot,2} = \frac{1}{mA} \left(\frac{w_{0,2} - w_c}{w_c - w_e} + \ln \frac{w_c - w_e}{w_2 - w_e} \right)$$

```
import numpy as np

w_e=0.02
w_c=0.18

w0_1=0.2
w_1=0.05
t_1=7300

mA=(1/t_1)*((w0_1-w_c)/(w_c-w_e)+np.log((w_c-w_e)/(w_1-w_e)))

w0_2 = 0.3
w_2 = 0.03

t_2=(1/mA)*((w0_2-w_c)/(w_c-w_e)+np.log((w_c-w_e)/(w_2-w_e)))

print("\nThe drying time is", f"{t_2:.4}", "[s]")
```

The drying time is 1.429e+04 [s]

Task 5.

One cannot plot the dependence of the time on m alone, but the term mA can be used instead.

```
import numpy as np
import matplotlib.pyplot as plt

# Data
N=1000
mA_v = np.linspace(mA/10, 10*mA, N)
t_v=(1/mA_v)*((w0_2-w_c)/(w_c-w_e)+np.log((w_c-w_e)/(w_2-w_e)))

#Plotting
figure=plt.figure()
```

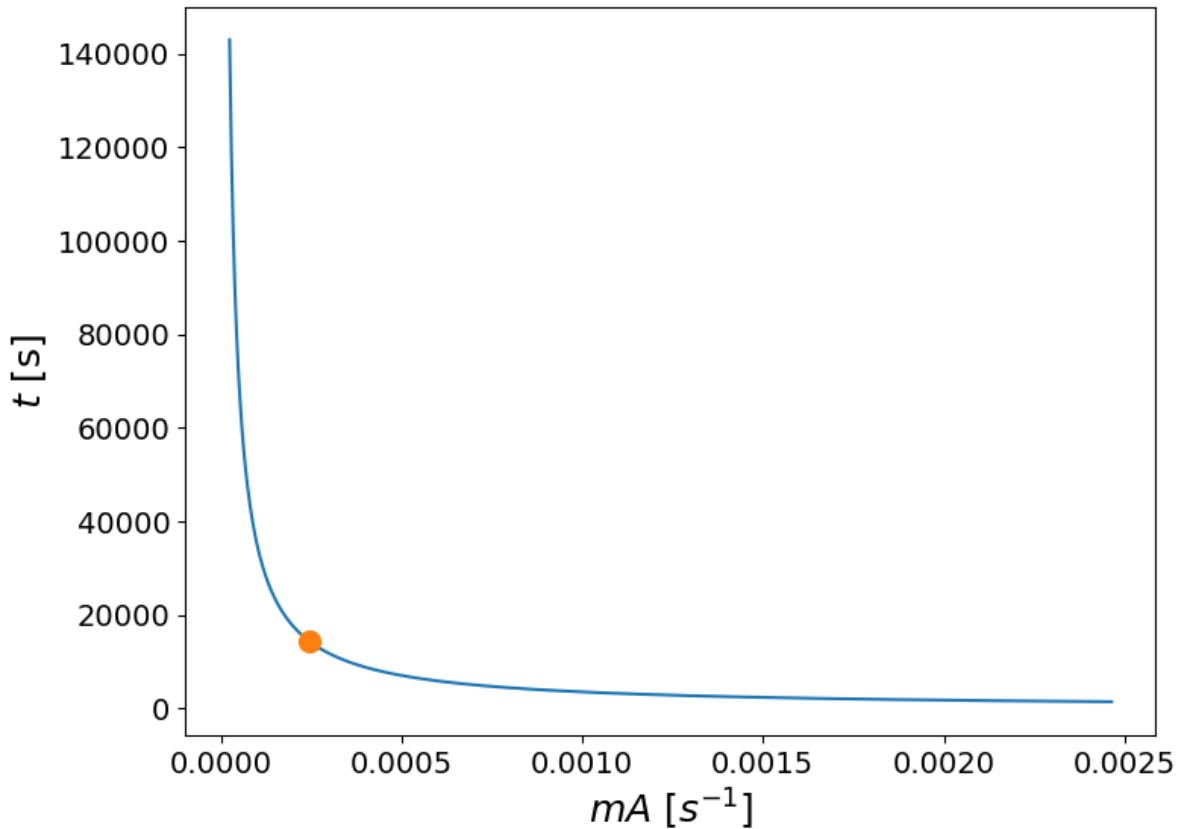
(continues on next page)

(continued from previous page)

```

axes = figure.add_axes([0.1,0.1,1,1])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
axes.plot(mA_v,t_v, marker=' ')
axes.plot(mA,t_2,marker='o',markersize=10)
axes.set_xlabel('$mA$ [$s^{-1}]$', fontsize=18);
axes.set_ylabel('$t$ [s]',fontsize=18);

```



0.18.4 Additional info

1. Asymptotic behaviour of the time for the moisture content reaching the equilibrium moisture content:

```

import numpy as np
import matplotlib.pyplot as plt

# Data
N=1000
ww = np.linspace(w_e*1.01,w_c*0.99, N)
t_div=(1/mA)*((w_0_2-w_c)/(w_c-w_e)+np.log((w_c-w_e)/(ww-w_e)))

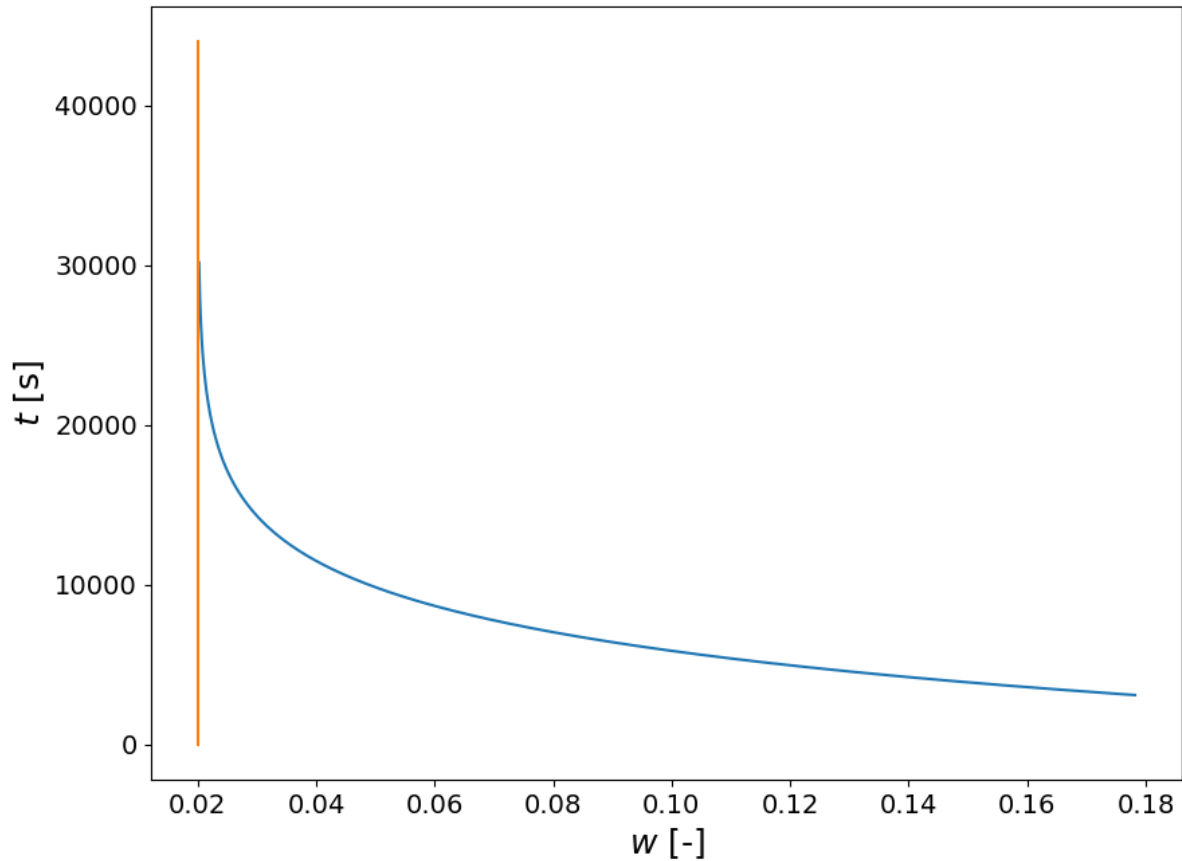
#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)

```

(continues on next page)

(continued from previous page)

```
plt.yticks(fontsize=14)
axes.plot(ww, tdiv, marker=' ')
axes.plot(np.array([we, we]), np.array([0, 44000]))
axes.set_xlabel('$w$ [-]', fontsize=18);
axes.set_ylabel('$t$ [s]', fontsize=18);
```



0.18.5 Contributions

Nikita Gusev, 29 March 2021

0.19 Week 11 - Exercise

0.19.1 Problem Statement

Water (density 1000 kg m^{-3} , viscosity 1 mN s m^{-2}) is used to fluidise a bed of packed spheres characterised by a diameter of 0.5 mm and a density of 1500 kg m^{-3} .

Tasks

- Demonstrate that, in laminar regime, the minimum fluidization velocity is always smaller than the terminal falling velocity.
- Compute the minimum fluidisation velocity and the terminal falling velocity of the bed particles.
- Sketch a plot of the ΔP as a function of the fluid velocity, clearly indicating the minimum fluidisation velocity.
- How would your result change if particles were twice as large?
- What does the terminal falling velocity represent in the context of fluidisation?

Solution

Task 1

In the laminar regime, for spherical particles of diameter d the minimum fluidization velocity is given by:

$$v_{mf} = \frac{1}{180} \frac{e^3}{(1-e)} \frac{d^2(\rho_s - \rho_f)g}{\mu}$$

where, e is the void fraction, d the diameter of the particles, ρ_s is the density of the particles, ρ_f is the density of the fluidising fluid, g is the gravity acceleration constant, and μ the viscosity of the fluid.

The terminal falling velocity, conversely is determined by the steady state force balance on a single particle:

$$v_t^2 = Cd^{-1} \frac{(\rho_s - \rho_f)}{\rho_f} \frac{4d}{3} g$$

where Cd is the drag coefficient.

If we introduce the definition of the drag coefficient in laminar regime given by the Stokes relation:

$$Cd = \frac{24}{Re_p} = \frac{24\mu}{\rho_f dv}$$

where Re_p is the Reynolds number and v the velocity of the fluid, we get:

$$v_t = \frac{\rho_f d}{24\mu} \frac{(\rho_s - \rho_f)}{\rho_f} \frac{4d}{3} g = \frac{1}{18} \frac{d^2(\rho_s - \rho_f)g}{\mu}$$

Now, if we take the ratio between v_{mf} and v_t we get:

$$\frac{v_{mf}}{v_t} = \frac{\frac{1}{180} \frac{e^3}{(1-e)} \frac{d^2(\rho_s - \rho_f)g}{\mu}}{\frac{1}{18} \frac{d^2(\rho_s - \rho_f)g}{\mu}} = \frac{1}{10} \frac{e^3}{(1-e)}$$

Since the void fraction $0 < e < 1$ by definition, with a lower theoretical limit at ≈ 0.36 (corresponding to a close-packing of equal spheres), the term $\frac{e^3}{(1-e)}$ is of order e^2 and smaller than 1.

Hence

$$\frac{v_{mf}}{v_t} < 1$$

and the minimum fluidization velocity is always smaller than the terminal Falling velocity, irrespective of d , ρ_p , ρ_f or μ .

Task 2 / 4

In order to solve task 2 we need to estimate the void fraction. A typical approach when the actual void fraction is not provided is to compute it as the ratio between the volume of a sphere of diameter d and a cube with the side of length d .

$$e = 1 - \frac{\pi}{6}$$

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

rho_f=1000 #
rho_s=1500 # kg m$^{-3}$

mu = 1E-3 # N s m$^{-2}$
d = 0.5 * 1E-3 #m
g=9.81 # m /s^2
e=1-np.pi/6
common=d**2*(rho_s-rho_f)*g/mu #m/s
v_mf=1/180*(e**3/(1-e))*common
v_t=1/18*common

print("\nDiameter: ", f"{d:.4}", " [mm]")
print("\nTerminal Falling velocity: ", f"{v_t:.4}", " [m/s]")
print("Minimum Fluidisation velocity: ", f"{v_mf:.4}", " [m/s]")

common=(2*d)**2*(rho_s-rho_f)*g/mu #m/s
v_mf=1/180*(e**3/(1-e))*common
v_t=1/18*common

print("\nDiameter: ", f"{2*d:.4}", " [mm]")
print("\nTerminal Falling velocity: ", f"{v_t:.4}", " [m/s]")
print("Minimum Fluidisation velocity: ", f"{v_mf:.4}", " [m/s]\n")
```

```
Diameter:  0.0005  [mm]

Terminal Falling velocity:  0.06813  [m/s]
Minimum Fluidisation velocity:  0.001407  [m/s]

Diameter:  0.001  [mm]

Terminal Falling velocity:  0.2725  [m/s]
Minimum Fluidisation velocity:  0.005627  [m/s]
```

Task 3 / 4

Without specifying the total length of the packed bed it is only possible to plot the pressure gradient $\Delta P/l$.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

# Data
N=1000
```

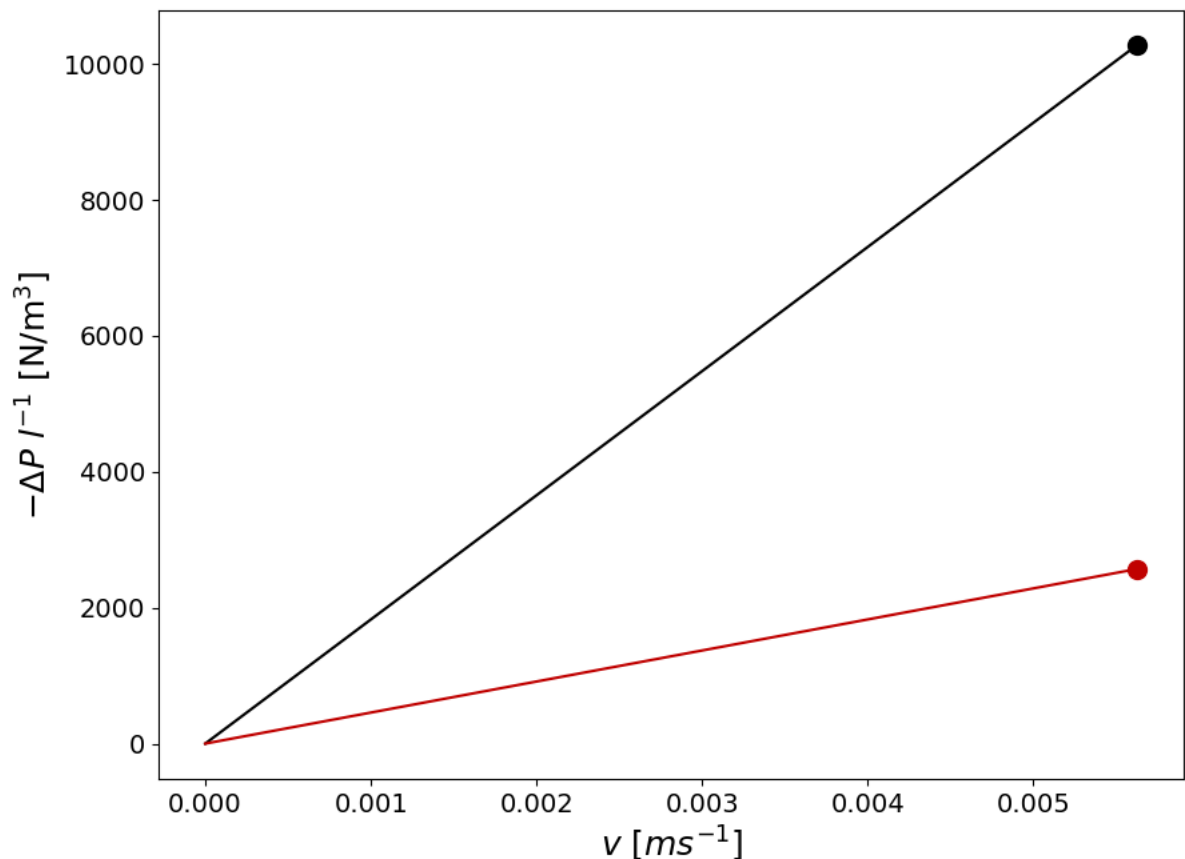
(continues on next page)

(continued from previous page)

```
v=np.linspace(0,v_mf, N)
color=iter(cm.gist_heat(np.linspace(0,1,3)))

#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
mDeltaP_l_mf=v_mf*180*(1-e)**2/e**3*mu/d**2 # N/m
mDeltaP_l=v*180*(1-e)**2/e**3*mu/d**2 # N/m
c=next(color)
axes.plot(v,mDeltaP_l, marker=' ',c=c)
axes.plot(v_mf,mDeltaP_l_mf,marker='o',markersize=10,c=c)

# Double particle diameter
c=next(color)
mDeltaP_l_mf=v_mf*180*(1-e)**2/e**3*mu/(2*d)**2 # N/m
mDeltaP_l=v*180*(1-e)**2/e**3*mu/(2*d)**2 # N/m
axes.plot(v,mDeltaP_l, marker=' ',c=c)
axes.plot(v_mf,mDeltaP_l_mf,marker='o',markersize=10,c=c)
axes.set_xlabel('$v$ [$m s^{-1}$]', fontsize=18);
axes.set_ylabel('$-\Delta P l^{-1}$ $[N/m^3]$', fontsize=18);
```



We can treat the bed length as a parameter, to show the behaviour of the ΔP as a function of velocity:

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.pyplot import cm

# Data
N=1000
#Plotting
figure=plt.figure()
axes = figure.add_axes([0.1,0.1,1.2,1.2])
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

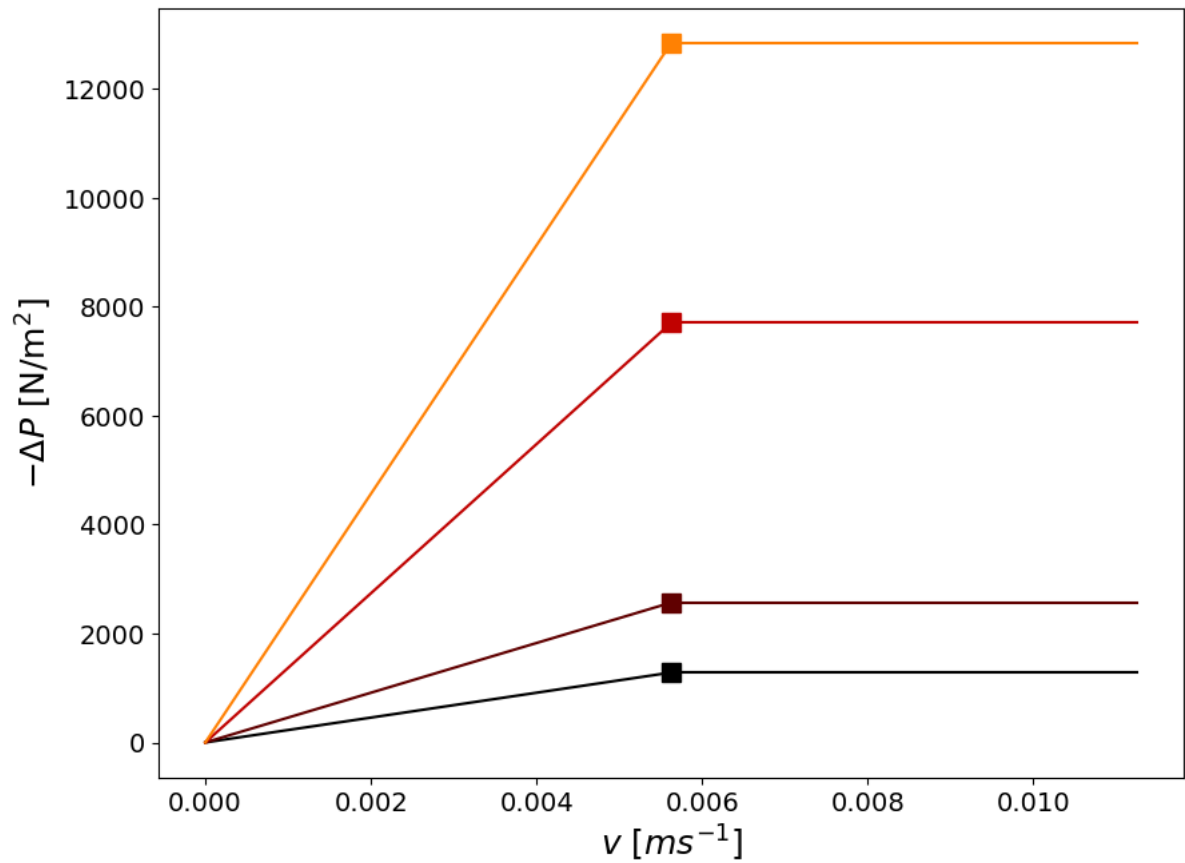
# Parameter: bed length
lengths=np.array([0.5, 1, 3, 5])

color=iter(cm.gist_heat(np.linspace(0,1,np.size(lengths)+1)))

for l in lengths:
    c=next(color)
    mDeltaP_mf=mDeltaP_l_mf*l # N/m
    mDeltaP=mDeltaP_l*l # N/m
    axes.plot(v,mDeltaP, marker=' ',c=c)
    axes.plot(np.array([v_mf, 2*v_mf]),mDeltaP_mf*np.array([1,1]), marker=' ',c=c)
    axes.plot(v_mf,mDeltaP_mf,marker='s',markersize=10,c=c)

axes.set_xlabel('$v$ [$m s^{-1}$]', fontsize=18);
axes.set_ylabel('$-\Delta P$ [N/m$^2$]', fontsize=18);

```



Task 5

The terminal falling velocity corresponds to the velocity at which the drag forces counterbalance the net weight of the particles, overcoming this velocity leads to net upward acceleration of the particles and corresponds to the instauration of the transport regime.